

UNIVERSIDAD SIMÓN BOLÍVAR

DECANATO DE ESTUDIOS PROFESIONALES

COORDINACIÓN DE INGENIERÍA DE COMPUTACIÓN

PROTOTIPO DE DETECCIÓN Y RESPUESTA ASISTIDA POR MODELOS DE LENGUAJE EN ENTORNOS EMPRESARIALES

Por:

Blanyer Michell Vielma Villarroel

INFORME DE PASANTÍA

Presentado ante la Ilustre Universidad Simón Bolívar

como requisito parcial para optar al título de

Ingeniero de Computación

Sartenejas, Febrero de 2027



UNIVERSIDAD SIMÓN BOLÍVAR

DECANATO DE ESTUDIOS PROFESIONALES

COORDINACIÓN DE INGENIERÍA DE COMPUTACIÓN

PROTOTIPO DE DETECCIÓN Y RESPUESTA ASISTIDA POR MODELOS DE LENGUAJE EN ENTORNOS EMPRESARIALES

Por:

Blanyer Michell Vielma Villarroel

Realizado con la asesoría de:

Tutor Académico: Prof. Oswaldo Monasterios Carrero

Tutor Industrial: David Enrique Altuve Villalba

INFORME DE PASANTÍA

Presentado ante la Ilustre Universidad Simón Bolívar

como requisito parcial para optar al título de

Ingeniero de Computación

Sartenejas, Febrero de 2027

[Espacio reservado para el Acta de Evaluación de Pasantía,
con las firmas originales del jurado.]

Resumen

Los sistemas de seguridad basados en reglas generan más alertas de las que un equipo puede analizar, muchas falsas y ninguna justificada. Este trabajo, realizado como pasantía larga en Domotai C.A., desarrolla un motor de triaje que clasifica y prioriza alertas, justifica cada decisión con un modelo de lenguaje (LLM) apoyado en generación aumentada por recuperación (RAG) en local, y propone una acción de respuesta contenida sujeta a validación humana: el modelo justifica, no decide. Se modeló un cliente genérico, se revisó el estado del arte, se construyó un laboratorio reproducible con Containerlab y Wazuh, se diseñó una arquitectura que separa clasificación, política determinista y perfil de cliente configurable, y se evaluó el prototipo frente al nivel de regla de Wazuh. Sobre 300 alertas reales de tres familias, incluidos casos diseñados para que las reglas fallaran, el prototipo redujo la tasa de falsos positivos a 0,009 frente a 0,310 del método por reglas y detectó todas las amenazas, incluidas las que el nivel de regla deja pasar (exhaustividad 0,920 del baseline), sin acciones disruptivas. Un árbol de decisión entrenado sobre esos datos redescubrió las reglas escritas a mano y aportó una, la de la ráfaga, adoptada como regla auditable. La recuperación aumentada corrigió la imprecisión de las explicaciones. Queda como limitación el clasificador con ajuste fino, que exige más variedad de datos de la que el laboratorio produce.

Palabras clave: triaje de alertas de seguridad, modelos de lenguaje de gran escala, generación aumentada por recuperación, MITRE ATT&CK, validación humana.

ÍNDICE GENERAL

Resumen **iv**

Índice de tablas **viii**

Índice de figuras **ix**

Lista de símbolos **x**

Lista de abreviaturas **xi**

Introducción **1**

1. Descripción de la empresa **6**

1.1. Identificación y ubicación 6

1.2. Actividad y servicios 6

1.3. Modelo de trabajo 7

1.4. Ámbito de la pasantía 7

2. Marco teórico **9**

2.1. El problema del triaje de alertas de seguridad 9

2.2. Detección y respuesta: EDR, XDR y MDR 10

2.3. MITRE ATT&CK como taxonomía de referencia 10

2.4. Modelos de lenguaje de gran escala aplicados a la seguridad 11

2.5. Generación aumentada por recuperación 12

2.6. Agentes con herramientas y el patrón de razonar y actuar 13

2.7. Riesgos de aplicar un modelo de lenguaje al triaje de seguridad 14

2.8. Reglas y firmas frente a análisis asistido por modelos de lenguaje 15
2.9. Políticas de decisión determinista y separación entre analizar y actuar 16
2.10. Métricas de evaluación de un clasificador binario 16
2.11. Entornos de red virtualizados para experimentación en seguridad 17

3. Metodología **18**

3.1. Enfoque general: modelar en lugar de esperar 18
3.2. Fase 1: modelado del cliente y del caso de uso 18
3.3. Fase 2: estado del arte y registro de requisitos 19
3.4. Fase 3: construcción del entorno de pruebas 19
3.5. Fase 4: diseño de la arquitectura 21
3.6. Fase 5: implementación 22
3.7. Fase 6: evaluación 26

4. Resultados y discusión **27**

4.1. Resultado principal: clasificación frente al método de referencia 27
4.2. La progresión de la medida, y por qué se reporta completa 28
4.3. Por qué el óptimo del baseline es débil 28
4.4. Agrupación por incidente: menos elementos que revisar 29
4.5. Operación y continuidad del servicio 29
4.5.1. Ampliación posterior: continuidad bajo un atacante interno 30
4.6. Calidad de la justificación: anclada, pero inicialmente poco fiable 31
4.7. El contraste con recuperación aumentada 31
4.8. Revisión de la invocación y medición con un modelo mayor 32
4.8.1. La recuperación mejora los verdaderos positivos y empeora los falsos . 33
4.8.2. La corrección, y el intercambio que revela 34
4.8.3. Cerrar el circuito: recuperar el motivo del descarte 35
4.8.4. El agente de mitigación, medido en condiciones válidas 36
4.9. La recuperación: un techo, su causa y su desplazamiento 37

4.9.1. Con un modelo de *embeddings* dedicado 38
4.9.2. Reproducibilidad de la generación (RNF-03) 39
4.10. Verificación sobre el conjunto completo 40
4.11. Segunda familia de ataque: reconocimiento 41
4.11.1. Tercera familia: el servicio en claro, y el nivel de regla pierde amenazas 43
4.12. El clasificador entrenado, y lo único que aportó 43
4.13. Limitaciones y análisis de errores 45

Conclusiones y recomendaciones **47**

Referencias **53**

ÍNDICE DE TABLAS

1.1. Líneas de servicio de Domotai C.A. 7

2.1. Reglas y firmas frente a análisis asistido por modelos de lenguaje 15

2.2. Métricas de clasificación empleadas en la evaluación 17

4.1. Prototipo frente al baseline de Wazuh (partición de evaluación, $n = 205$) . . 27

4.2. Medida F_1 del baseline según el umbral de nivel de regla 29

4.3. Reducción de elementos a revisar por agrupación en incidentes 29

4.4. Justificación con el modelo pequeño y con uno mayor, misma partición . . . 33

4.5. Los tres estados de la justificación con recuperación, misma partición y mismo modelo 34

4.6. Efecto de ampliar el corpus sobre la calidad de la recuperación 38

4.7. Calidad de la recuperación según cómo se formule la consulta 38

4.8. Calidad de la recuperación según el modelo de *embeddings*, consulta fija . . . 39

4.9. Clasificación sobre la partición ampliada ($n = 220$, 31 soportadas), antes y después de declarar al auditor como origen legítimo 41

ÍNDICE DE FIGURAS

3.1. Topología del laboratorio 20

3.2. Arquitectura del motor de triaje 21

Lista de símbolos

Símbolos latinos

F_1	Media armónica de la precisión y la exhaustividad
FN	Número de falsos negativos de la matriz de confusión
FP	Número de falsos positivos de la matriz de confusión
k	Número de pasajes recuperados por consulta en el módulo de generación aumentada por recuperación
n	Número de alertas de una muestra o de una campaña de evaluación
P	Precisión de la clasificación
R	Exhaustividad (*recall*) de la clasificación
VN	Número de verdaderos negativos de la matriz de confusión
VP	Número de verdaderos positivos de la matriz de confusión

Lista de abreviaturas

Sigla	Significado
ATT&CK	*Adversarial Tactics, Techniques, and Common Knowledge* — matriz de tácticas y técnicas adversarias de MITRE
BERT	*Bidirectional Encoder Representations from Transformers* — modelo de representación de lenguaje preentrenado
CART	*Classification and Regression Trees* — árboles de clasificación y regresión
CLS	Token de clasificación (*classification*) de un modelo tipo BERT
EDR	*Endpoint Detection and Response* — detección y respuesta en el *endpoint*
FN	Falso negativo
FP	Falso positivo
GPU	*Graphics Processing Unit* — unidad de procesamiento gráfico
IoT	*Internet of Things* — internet de las cosas
IP	*Internet Protocol* — protocolo de internet
LLM	*Large Language Model* — modelo de lenguaje de gran escala
MDR	*Managed Detection and Response* — detección y respuesta gestionada
OWASP	*Open Web Application Security Project* — proyecto abierto de seguridad en aplicaciones web
RAG	*Retrieval-Augmented Generation* — generación aumentada por recuperación
RF	Requisito funcional
RNF	Requisito no funcional
SIEM	*Security Information and Event Management* — gestión de información y eventos de seguridad
SOC	*Security Operations Center* — centro de operaciones de seguridad
SSH	*Secure Shell* — protocolo de administración remota cifrada
USB	Universidad Simón Bolívar
VN	Verdadero negativo
VP	Verdadero positivo

Sigla	Significado
XDR	*Extended Detection and Response* — detección y respuesta extendida

Introducción

Los equipos de operaciones de seguridad reciben, en la práctica totalidad de las organizaciones, un volumen de alertas que supera su capacidad de análisis manual. La causa dominante no es la falta de detección, sino el exceso de ella: los sistemas de correlación basados en reglas y firmas —el fundamento de la mayoría de las plataformas de detección y respuesta hoy desplegadas— generan un alto volumen de falsos positivos, y cada uno exige el mismo esfuerzo de interpretación que una amenaza real, porque la alerta señala qué regla se disparó, no por qué eso importa en el contexto concreto del activo afectado.

El presente trabajo, realizado como pasantía larga en la empresa Domotai C.A., aborda ese problema mediante un motor de triaje que clasifica y prioriza alertas de seguridad de red, justifica cada decisión con un modelo de lenguaje de gran escala (LLM, del inglés *Large Language Model*) apoyado en generación aumentada por recuperación (RAG, del inglés *Retrieval-Augmented Generation*) ejecutada en un entorno local, y propone una acción de respuesta contenida sujeta a validación humana antes de ejecutarse.

Antecedentes

Las plataformas comerciales de detección y respuesta han evolucionado del *endpoint* aislado —EDR, del inglés *Endpoint Detection and Response*— hacia la correlación multicapa que combina red, identidad, correo y nube en una sola plataforma, conocida como XDR (*Extended Detection and Response*); y hacia el servicio operado por un equipo humano que las administra las veinticuatro horas, conocido como MDR (*Managed Detection and Response*). Las evaluaciones independientes de estas plataformas —notablemente las de MITRE ATT&CK Evaluations, que mide su cobertura de técnicas adversarias contra la matriz pública MITRE ATT&CK— siguen registrando de forma explícita el volumen de alertas que un analista debe triar como un criterio de comparación entre proveedores, lo que confirma que la fatiga de falsos positivos persiste incluso en las soluciones líderes del mercado.

En paralelo, los modelos de lenguaje de gran escala han empezado a incorporarse a la operación de seguridad. Una primera línea de trabajo continuó el preentrenamiento de

modelos de tipo *encoder* sobre corpus específicos de ciberseguridad —informes de amenazas, bases de vulnerabilidades, documentación técnica—, de lo que **SecureBERT** es la referencia original [1]. Una segunda línea publicó modelos generativos con preentrenamiento continuado sobre el mismo tipo de corpus, de los que **Foundation-Sec-8B**, de Cisco Foundation AI, es la referencia pública más reciente, con variantes que declaran explícitamente el triaje y el resumen de casos de un centro de operaciones de seguridad (SOC, *Security Operations Center*) como caso de uso objetivo [2]. Las plataformas comerciales líderes —Microsoft Security Copilot, CrowdStrike Charlotte AI y equivalentes— han incorporado asistentes basados en estos modelos, con un patrón consistente: redactan y resumen, no deciden; la autoridad final permanece humana.

Complementando a los modelos generativos, la generación aumentada por recuperación demostró que combinar un modelo de lenguaje con un sistema de recuperación sobre un corpus externo permite anclar sus respuestas a documentos concretos y actualizar ese conocimiento sin reentrenar el modelo [3]. Esta técnica es la que este trabajo aplica para corregir la principal limitación detectada en la justificación generada por un modelo pequeño: describe correctamente qué campos cita, pero se equivoca en lo que esos campos significan.

Justificación e importancia

Adoptar un modelo de lenguaje en el camino de una alerta de seguridad no es una decisión neutra. El catálogo de riesgos de OWASP para aplicaciones basadas en LLM sitúa la inyección de instrucciones como el riesgo número uno [4], y en este dominio no es un riesgo hipotético: el contenido de un registro de eventos —el nombre de usuario probado, la cadena completa del evento— lo escribe quien está atacando. Si esos campos se concatenan al mensaje del modelo sin distinguirlos de una instrucción, el atacante obtiene un canal directo hacia el sistema que decide sobre su propio ataque. A esto se suma un segundo riesgo, más sutil: la alucinación con apariencia de fundamento, en la que el modelo redacta una explicación plausible que cita un dato que la alerta no contiene, y que un analista con prisa acepta precisamente porque suena a análisis correcto.

Estos riesgos, junto con la exigencia de que el análisis de una alerta no salga de la red del cliente hacia un servicio externo, justifican dos decisiones de diseño que atraviesan todo el trabajo: que los campos de una alerta se traten siempre como datos y nunca como instrucciones, y que el modelo se ejecute en un entorno local, sin depender de una nube de terceros. La segunda decisión responde además a una necesidad de negocio concreta de la empresa: Domotai C.A. ofrece la automatización asistida por inteligencia artificial como una de sus líneas de servicio, y un módulo que reduzca el ruido operativo de un cliente sin exponer sus datos ocupa un lugar dentro de esa oferta que las plataformas comerciales, todas ellas

dependientes de la nube del proveedor, no cubren.

Por último, la restricción de continuidad de servicio es la que distingue a este trabajo de un sistema de respuesta automatizada convencional: un triaje que acierta más pero que, por error, interrumpe un servicio que el cliente presta a los suyos es peor que el sistema al que sustituye. De ahí que la decisión de qué acción ejecutar nunca la tome el propio modelo de lenguaje, sino una política determinista filtrada por un perfil de cliente configurable, con la validación de una persona antes de cualquier acción que alcance un servicio.

Planteamiento del problema

Diseñar este motor de triaje sobre el sistema de análisis de alertas que la empresa usa hoy exigía partir de tres elementos que, en la práctica, no estaban disponibles al inicio de la pasantía: el propio sistema de logs y correlación de un cliente real, un entorno donde ejercitarlo sin riesgo para una operación productiva, y un conjunto de alertas con verdaderos y falsos positivos identificados contra los que medir cualquier mejora. Sin una fuente de alertas no hay nada que triar; sin un entorno controlado no hay dónde ejecutar ni verificar una acción de respuesta; y sin un conjunto etiquetado no es posible calcular precisión, exhaustividad ni ninguna otra métrica de clasificación.

El problema, formulado de manera acotada, es entonces el siguiente: dado que no se dispone de acceso a un cliente ni a su módulo propietario de análisis de alertas, ¿es posible construir, evaluar y demostrar en un entorno de laboratorio reproducible un motor de triaje que clasifique alertas de acceso no autorizado a servicios de gestión expuestos, justifique cada decisión con razonamiento verificable en vez de con la sola cita de una regla, y proponga una acción de respuesta acotada sin comprometer la continuidad del servicio que protege, de forma que el resultado sea representativo de lo que ese motor haría sobre la infraestructura real de un cliente cuando esta esté disponible?

La respuesta de diseño fue modelar un cliente genérico en vez de esperar a uno concreto, y sustituir el sistema de logs inexistente por Wazuh, una plataforma de detección basada en reglas desplegada dentro de un laboratorio de red emulado con Containerlab [5], cuyo nivel de regla cumple además el papel de método tradicional de referencia contra el que se mide el prototipo.

Objetivo general

Desarrollar un prototipo para la detección y el triaje de alertas de seguridad en entornos empresariales, basado en modelos de lenguaje con un módulo de generación aumentada por recuperación ejecutado sobre un modelo desplegado de forma local, de manera que permita

clasificar y priorizar alertas con razonamiento explicable, reduciendo los falsos positivos y apoyando la labor de los analistas de seguridad.

Objetivos específicos

1. **Analizar y aprender a usar el módulo propietario.** Analizar la solución actual de la empresa para la recepción y el análisis de alertas, identificar sus limitaciones y las necesidades no cubiertas, y definir el caso de uso acotado sobre el que se construiría el prototipo.

2. **Revisar el estado del arte.** Analizar las soluciones de detección y respuesta extendida y gestionada existentes en el mercado y las aplicaciones de modelos de lenguaje en el ámbito de la ciberseguridad, identificando sus ventajas, sus limitaciones y las necesidades no cubiertas por las herramientas actuales, para derivar de ello los requisitos funcionales y no funcionales del prototipo.

3. **Configurar un entorno de pruebas.** Instalar y poner en funcionamiento un entorno con casos de prueba representativos, garantizando que los datos no salgan hacia servicios externos, y preparar un conjunto de alertas de prueba etiquetado —verdaderos y falsos positivos— que sirviera de referencia de evaluación.

4. **Diseñar la arquitectura del sistema.** Definir el flujo de procesamiento, las reglas de decisión y sus umbrales, la forma de interacción con el entorno de pruebas, el formato del razonamiento explicable y los puntos de validación humana, así como las métricas de evaluación y el método tradicional de referencia contra el que se compararía el prototipo.

5. **Implementar el prototipo.** Construir el caso de uso acotado, integrando la ingesta y normalización de alertas, la clasificación y priorización, la generación de justificación explicable y el flujo de validación humana en las decisiones críticas, con registro de trazas para auditoría.

6. **Evaluar el prototipo.** Medir el desempeño del sistema frente al método tradicional de referencia definido en el objetivo anterior, y analizar los errores sistemáticos y las limitaciones del enfoque.

7. **Elaborar la documentación técnica.** Generar la documentación y el informe técnico con la arquitectura, las decisiones de diseño, los resultados, las limitaciones y las líneas de trabajo futuro.

Estructura del informe

El documento se organiza en cuatro capítulos. El primero describe la empresa donde se desarrolló la pasantía. El segundo reúne el marco teórico, recorriendo desde el problema del triaje de alertas y las plataformas de detección y respuesta hasta los modelos de lenguaje, la generación aumentada por recuperación, los agentes con herramientas, los riesgos de aplicar todo ello a este dominio y las métricas con que se evalúa. El tercero expone la metodología seguida a lo largo de las siete fases del proyecto, y el cuarto presenta los resultados obtenidos y su discusión. Cierran el documento las conclusiones y recomendaciones, y las referencias consultadas.

CAPÍTULO 1

DESCRIPCIÓN DE LA EMPRESA

La pasantía se desarrolla en Domotai C.A., una empresa venezolana de desarrollo de software enfocada en la aplicación de la inteligencia artificial a productos y procesos. Este capítulo describe su actividad, sus líneas de servicio y su forma de trabajo, y sitúa el proyecto de la pasantía dentro de ese contexto.

1.1. Identificación y ubicación

Domotai C.A. opera desde el Edificio Caracas Campus, en la calle Altagracia, urbanización La Trinidad, municipio Baruta del estado Miranda, en la ciudad de Caracas. Su actividad no se limita al mercado local: atiende clientes de sectores diversos bajo un modelo de trabajo que no exige la presencia física del cliente en sus instalaciones.

1.2. Actividad y servicios

La empresa combina el criterio de sus profesionales con herramientas de inteligencia artificial para el desarrollo de productos de software. Su oferta se organiza en las líneas recogidas en la tabla 1.1.

Tabla 1.1. Líneas de servicio de Domotai C.A.

Línea	**Alcance**
Desarrollo web	Sitios corporativos y aplicaciones de cliente desarrollados a medida
Desarrollo móvil	Aplicaciones multiplataforma
Automatización con inteligencia artificial	Agentes y flujos automatizados integrados en los procesos del cliente
Integración *hardware-software*	Soluciones que articulan componentes físicos con sistemas informáticos
Diseño de interfaz y experiencia de usuario	Definición de la interacción y del aspecto visual de los productos
Asesoría tecnológica	Acompañamiento en decisiones de arquitectura y selección de herramientas

De estas líneas, la automatización mediante inteligencia artificial es la que enmarca directamente el proyecto de esta pasantía y a la que se vuelve al cierre del capítulo.

1.3. Modelo de trabajo

Domotai C.A. acumula más de veinte proyectos completados, con un plazo de entrega característico del orden de seis semanas: un modo de operar orientado a ciclos cortos, en los que el alcance del producto se acota para poder entregarse dentro de ese plazo en lugar de plantearse como un desarrollo prolongado. Su cartera abarca dominios heterogéneos —diseño y arte, hostelería, interiorismo, comercio electrónico—, lo que indica que su especialización no reside en un sector vertical concreto, sino en un conjunto de capacidades técnicas aplicables a dominios distintos.

La contratación se articula bajo dos modalidades: por proyecto, con un alcance cerrado de antemano, y por suscripción, que da al cliente acceso continuado a la capacidad de desarrollo de la empresa. La segunda modalidad supone una relación sostenida en el tiempo, lo que favorece la acumulación de conocimiento sobre el dominio de cada cliente.

1.4. Ámbito de la pasantía

El proyecto desarrollado durante esta pasantía se sitúa en la línea de automatización con inteligencia artificial descrita en la tabla 1.1, aplicada esta vez al dominio de la ciberseguridad:

un motor que triaje alertas de seguridad de red, las clasifique y priorice, y justifique cada decisión mediante un modelo de lenguaje con generación aumentada por recuperación, en lugar de mediante trabajo manual especializado de un analista.

El encaje con la empresa es directo por dos razones. La primera es de mercado: Domotai C.A. no compite en un segmento vertical fijo, sino que ofrece capacidades de automatización aplicables a dominios distintos, y la seguridad de red —con su necesidad de reducir el trabajo manual repetitivo de un especialista— es uno de esos dominios. La segunda es de restricción compartida: al igual que en el resto de su cartera, la solución debía operar sin depender de servicios en la nube de terceros, de forma que los datos del cliente —en este caso, sus alertas de seguridad— no salgan de su propia red.

La supervisión de la pasantía corresponde a David Enrique Altuve Villalba, Gerente de Operaciones de la compañía, designado tutor industrial. La estancia se desarrolla en modalidad presencial en la sede de la empresa, con la duración de veinte semanas que corresponde a una pasantía larga de la Universidad Simón Bolívar.

CAPÍTULO 2

MARCO TEÓRICO

Este capítulo reúne los fundamentos sobre los que se apoya el desarrollo. Avanza desde el problema que motiva el proyecto —la fatiga de falsos positivos en el triaje de alertas— hacia las plataformas que hoy lo abordan, los modelos de lenguaje que este trabajo incorpora, la técnica de recuperación que corrige su principal debilidad, los riesgos de aplicarlos a este dominio, el razonamiento que sostiene que la decisión final no debe tomarla el modelo, y por último los criterios con que se mide el resultado.

2.1. El problema del triaje de alertas de seguridad

Un sistema de detección basado en reglas y firmas dispara una alerta cuando una secuencia de eventos coincide con un patrón previamente definido: una autenticación fallida, un puerto que empieza a recibir tráfico, un proceso que no debería ejecutarse. Esa coincidencia es determinista y auditable —se puede señalar exactamente qué regla se disparó—, pero no incorpora ningún juicio sobre si, en el contexto del activo concreto que la generó, esa coincidencia representa una amenaza real. La misma regla se dispara igual sobre un servidor expuesto a internet que sobre uno que no lo está, y el resultado es un volumen de falsos positivos que crece con el tamaño de la red sin que crezca en la misma proporción el riesgo real.

El *triaje* es la tarea de decidir, para cada alerta, si merece la atención de un analista y con qué prioridad. Cuando esa tarea se realiza manualmente sobre un volumen de alertas que supera la capacidad del equipo, dos fallos son posibles y ambos son costosos: tratar como amenaza algo que no lo es, que consume tiempo del analista sin reducir el riesgo, y descartar como ruido algo que sí lo es, que es exactamente el fallo que un sistema de seguridad no puede permitirse. Las evaluaciones independientes de las plataformas comerciales de detección más recientes miden de forma explícita el volumen de alertas que un analista debe procesar como un criterio de comparación entre proveedores [6], lo que confirma que este problema no es una limitación de las herramientas más antiguas, sino una característica persistente del propio enfoque basado en reglas, incluso en sus implementaciones más avanzadas.

2.2. Detección y respuesta: EDR, XDR y MDR

Tres siglas dominan el mercado de plataformas de detección y conviene distinguirlas, porque operan en planos distintos.

EDR (*Endpoint Detection and Response*, detección y respuesta en el *endpoint*) es la tecnología que recoge telemetría de un equipo concreto —procesos, ficheros, registro, conexiones de red locales— y responde sobre ese mismo equipo: aislarlo, terminar un proceso, ponerlo en cuarentena.

XDR (*Extended Detection and Response*, detección y respuesta extendida) correlaciona telemetría de varias capas —*endpoint*, red, identidad, correo, nube— en una sola plataforma analítica. El EDR es, en este esquema, una de las fuentes que alimentan al XDR; lo que este añade es la correlación entre fuentes que, por separado, solo verían una parte del incidente.

MDR (*Managed Detection and Response*, detección y respuesta gestionada) no es una tecnología sino un servicio: un equipo humano que opera un EDR o un XDR por cuenta del cliente, haciendo el triaje, la investigación y la respuesta que el cliente no tiene capacidad de operar por sí mismo. Un MDR que opera sobre un XDR se denomina en ocasiones MXDR (*Managed XDR*).

Las plataformas comerciales líderes despliegan típicamente un agente propietario a nivel de sistema operativo, más rico que las fuentes de telemetría de código abierto —como Sysmon, osquery o Zeek— y con capacidad de respuesta activa integrada. Esas fuentes abiertas, sin embargo, no quedan excluidas del ecosistema: numerosas plataformas las ingieren como fuente complementaria en su capa de agregación, y los servicios MDR que no exigen un agente comercial concreto —el modelo que en este trabajo se denomina telemetría-agnóstico— operan directamente sobre la telemetría que el cliente ya tenga. Este proyecto se inserta precisamente en ese segundo modelo: no sustituye al agente ni a la fuente de telemetría, sino que añade la capa de clasificación, priorización y justificación explicable sobre las alertas que esa fuente ya produce.

2.3. MITRE ATT&CK como taxonomía de referencia

MITRE ATT&CK (*Adversarial Tactics, Techniques, and Common Knowledge*) es una matriz pública y de acceso libre que cataloga las tácticas y técnicas que un atacante emplea a lo largo de un incidente, desde el acceso inicial hasta la exfiltración, con un identificador único por técnica —por ejemplo, T1110 para el acceso por fuerza bruta, con la subtécnica T1110.001 para el intento por diccionario de contraseñas [7]. Su valor para este trabajo es doble: sirve como vocabulario común para expresar la hipótesis de ataque dentro de la justificación de

cada decisión, y sirve como criterio de comparación en las evaluaciones independientes de plataformas comerciales citadas en la sección anterior. MITRE mantiene además ATLAS, una matriz equivalente centrada en las amenazas contra los propios sistemas de inteligencia artificial [8], pertinente en la medida en que este trabajo introduce uno de esos sistemas en el propio camino de decisión.

2.4. Modelos de lenguaje de gran escala aplicados a la seguridad

Un modelo de lenguaje de gran escala (LLM) es una red neuronal entrenada para predecir la continuación más probable de una secuencia de texto, sobre un volumen de parámetros que va de varios cientos de millones a varios cientos de miles de millones. Aplicado a la seguridad, conviene distinguir tres formas de uso, que no compiten entre sí sino que resuelven partes distintas del problema de triaje:

- **Codificador clasificador** (*encoder*, con arquitectura de tipo BERT y del orden de cien millones de parámetros): asigna una clase y una confianza numérica a una entrada, tras un ajuste fino sobre datos etiquetados. Su coste de ejecución es muy bajo —del orden de milisegundos sobre un procesador convencional— y es determinista, pero no redacta ninguna explicación y no generaliza bien fuera de lo que vio durante el entrenamiento.
- **Modelo generativo instruido** (de varios miles de millones de parámetros): redacta el razonamiento, resume un incidente o traduce una consulta en lenguaje natural. Su coste es alto —segundos y varios gigabytes de memoria— y no es fiable como clasificador numérico ni determinista por defecto.
- **Modelo especializado de dominio**: un codificador o un generativo con preentrenamiento continuado sobre corpus de ciberseguridad. **SecureBERT** demostró que esa especialización mejora las tareas de clasificación del dominio frente al modelo genérico equivalente [1], y **Foundation-Sec-8B**, un modelo generativo de ocho mil millones de parámetros con preentrenamiento continuado sobre corpus de inteligencia de amenazas, declara explícitamente el triaje de un centro de operaciones de seguridad como uno de sus casos de uso objetivo [2]. La especialización aporta vocabulario —el modelo reconoce sin explicación previa un identificador de vulnerabilidad o una técnica ATT&CK— pero no aporta garantías: sigue pudiendo afirmar con aparente seguridad algo que la entrada no contiene.

La conclusión que este marco teórico traslada al diseño del prototipo es que clasificar y justificar son tareas de naturaleza distinta y conviene resolverlas con componentes distintos: un codificador da una clase estable, barata y con una confianza calibrable; un modelo generativo

produce el texto que una persona necesita leer para decidir. Pedirle ambas cosas al mismo componente tiende a producir puntuaciones de confianza inventadas con apariencia de cálculo.

2.5. Generación aumentada por recuperación

La generación aumentada por recuperación (RAG) combina un modelo de lenguaje con un mecanismo de recuperación de información sobre un corpus documental externo: ante una consulta, el sistema recupera primero los fragmentos de texto más relevantes de ese corpus y se los entrega al modelo como parte de su contexto, antes de que este redacte la respuesta [3]. Dos consecuencias hacen de esta técnica algo más que un atajo de ingeniería. La primera es que el conocimiento deja de residir exclusivamente en los parámetros del modelo —fijados en el momento de su entrenamiento— y pasa a un índice que puede actualizarse sin reentrenar nada. La segunda es que la respuesta queda *anclada*: cada afirmación puede, en principio, rastrearse hasta el fragmento del corpus que la sustenta, lo que reduce —sin eliminar— el riesgo de que el modelo invente un dato plausible.

La forma más simple de recuperación —la empleada en este trabajo— representa cada documento del corpus y cada consulta como un vector numérico mediante un modelo de *embeddings*, y ordena los documentos por su similitud coseno con la consulta:

$$\cos(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \, \|\mathbf{d}\|} \tag{2.1}$$

donde $\mathbf{q}$ es el vector de la consulta y $\mathbf{d}$ el de un documento del corpus; se seleccionan los k documentos de mayor similitud, con k típicamente pequeño (en este trabajo, $k = 3$) para no saturar el contexto del modelo con pasajes de relevancia marginal. Existen técnicas de recuperación más sofisticadas —dispersas, como BM25, o densas entrenadas específicamente para la tarea de recuperación, como en los sistemas de tipo *dense passage retrieval*— así como arquitecturas de recuperación por grafo; este trabajo emplea la variante más simple posible, por similitud de *embeddings* genéricos, dado que el objetivo no es optimizar la recuperación en sí sino verificar si corregir el conocimiento del modelo mejora la corrección de su justificación.

Un riesgo documentado de esta técnica, especialmente relevante cuando la consulta y el corpus se formulan en idiomas o registros distintos, es que la utilidad de un fragmento recuperado no es uniforme según su posición en el contexto que finalmente recibe el modelo: los fragmentos situados en el centro de un contexto largo se aprovechan peor que los situados en sus extremos [9]. Esta observación motivó, en el diseño del justificador de este trabajo, mantener el número de pasajes recuperados deliberadamente bajo.

2.6. Agentes con herramientas y el patrón de razonar y actuar

Un *agente* basado en un modelo de lenguaje es un modelo situado dentro de un bucle: en lugar de producir una única respuesta, alterna pasos de razonamiento con la invocación de herramientas externas, observa lo que estas devuelven y decide con esa información si continuar o concluir. La formulación que popularizó este esquema es el patrón conocido como *ReAct*, contracción de *reasoning and acting*, que entrelaza explícitamente las trazas de razonamiento con las acciones sobre el entorno y muestra que ambas se refuerzan: el razonamiento guía qué herramienta usar, y lo observado corrige el razonamiento siguiente [10].

Este trabajo emplea el patrón en dos puntos distintos, y conviene separarlos porque su riesgo no es el mismo. El primero es la **recuperación agéntica**: en vez de construir la consulta al corpus con una plantilla fija sobre los campos de la alerta, es el propio modelo el que formula qué necesita consultar antes de justificar. El segundo es el **agente de mitigación**, que ante un incidente cuya contención en el equipo afectado no basta —porque ese equipo está fuera de alcance— razona la escalada hacia el siguiente dispositivo con capacidad de contención. Este segundo uso no procede de la literatura sino de una necesidad operativa señalada por el tutor industrial de la empresa: en una red en producción el equipo atacado puede estar caído, comprometido o sin canal de administración, y quedarse sin capacidad de contención por ello no es aceptable.

La tensión con el resto de este marco teórico es evidente y merece enunciarse en lugar de disimularse: un agente que planifica libremente y encadena herramientas por su cuenta es precisamente lo que la exigencia de auditabilidad de la sección 2.9 rechaza. La forma de conservar la utilidad del patrón sin heredar ese problema es acotarlo por construcción, y en este trabajo se acota de tres maneras simultáneas: el agente **selecciona** dentro de un catálogo cerrado de acciones y nunca inventa una nueva; cada acción de la escalada pasa por el mismo filtro del perfil que el camino determinista —se aplica en automático solo lo que el perfil permite y se retiene para validación humana lo que este exige, y el agente no puede saltar a un cortafuegos perimetral sin haber intentado antes la contención en el equipo atacado—; y la decisión de contener —el qué— sigue siendo determinista, de modo que el agente elige la estrategia, no si hay que actuar. Un agente así no es autónomo en el sentido fuerte del término: es un selector razonado sobre un espacio de acciones cerrado y supervisado.

A esas tres cotas se añade una cuarta, de naturaleza distinta, que los motores de inferencia actuales permiten aplicar: la **generación restringida**. En lugar de pedir al modelo que respete un formato y comprobar después si lo hizo, se le impone una gramática —o un esquema de datos— durante el propio muestreo, de manera que todo token que violaría esa estructura recibe probabilidad nula. Si la gramática se construye a partir del catálogo de acciones y de la topología del cliente, el modelo deja de *poder* nombrar una herramienta, un dispositivo o

una acción que no existan. El catálogo cerrado pasa así de ser una validación posterior sobre texto libre a una imposibilidad estructural.

Conviene, sin embargo, enunciar con precisión qué garantiza esa técnica y qué no, porque la diferencia es fácil de pasar por alto y en este dominio es determinante: **restringir la salida garantiza que la forma sea válida, no que el contenido sea cierto**. Una gramática elimina la respuesta malformada; no elimina la afirmación falsa. Un modelo puede emitir un objeto perfectamente conforme al esquema declarando que ha contenido un incidente sin haber ejecutado ninguna acción. De ahí que la propiedad que realmente protege al sistema no sea sintáctica sino de procedencia: el resultado de una mitigación debe leerse del registro de lo que efectivamente se ejecutó y verificó, nunca de lo que el modelo afirma haber hecho.

2.7. Riesgos de aplicar un modelo de lenguaje al triaje de seguridad

Introducir un modelo de lenguaje en el camino de decisión de una alerta de seguridad tiene riesgos específicos de este dominio, más allá de los riesgos generales de cualquier aplicación basada en LLM catalogados por el proyecto abierto de seguridad en aplicaciones web [4] y por la evaluación de seguridad de modelos de Meta conocida como CyberSecEval [11].

Inyección de instrucciones desde campos controlados por el atacante. El contenido de un registro de eventos —el nombre de usuario probado, la cadena de *user-agent*, el mensaje completo del evento— lo escribe, en un intento de intrusión, la propia parte atacante. Si esos campos se concatenan sin distinguirlos de las instrucciones del sistema, el atacante dispone de un canal directo hacia el componente que analiza su propio ataque. La respuesta de diseño es tratar siempre esos campos como datos delimitados, nunca como instrucciones.

Alucinación con apariencia de fundamento. El fallo más peligroso no es el error evidente, sino la explicación bien redactada que cita un detalle que la entrada no contiene; un analista con prisa la acepta precisamente porque suena a análisis correcto. El contrapeso es exigir un anclaje verificable: toda afirmación debe poder rastrearse a un campo concreto de la alerta o del contexto recuperado, y esa verificación puede automatizarse.

Ausencia de determinismo. La misma entrada puede producir salidas distintas en dos ejecuciones sucesivas de un modelo generativo. Para la operación es una incomodidad menor; para la evaluación es descalificante, porque impide que los resultados sean reproducibles. Se controla fijando una temperatura de muestreo próxima a cero y registrando la versión exacta del modelo y de las instrucciones empleadas en cada ejecución.

Corte de conocimiento. Un modelo no sabe nada de lo publicado después de su entrenamiento, así que cualquier afirmación suya sobre el panorama de amenazas actual es

sospechosa por construcción; el conocimiento reciente debe entrar por el contexto que se le suministra, no salir de sus propios parámetros.

Superficie de fallo adicional. Un componente más que puede fallar, colgarse o rendir por debajo de lo esperado exige un mecanismo de degradación: si el modelo no responde o su respuesta no pasa la verificación de anclaje, la decisión debe seguir un camino alternativo —una plantilla determinista, en este trabajo— y nunca descartarse en silencio.

2.8. Reglas y firmas frente a análisis asistido por modelos de lenguaje

La tabla 2.1 resume la comparación entre ambos enfoques sobre los atributos que importan para el triaje.

Tabla 2.1. Reglas y firmas frente a análisis asistido por modelos de lenguaje

	Reglas y firmas	**Análisis asistido por modelo**
Decisión	Determinista y repetible	Probabilístico; repetible solo si se fuerza
Auditabilidad	Total: se lee la regla que se disparó	Indirecta: se lee la justificación, no el cálculo
Contexto del activo	Ninguno	Su mejor aportación, si se le suministra
Explicación	El nombre de la regla	Razonamiento en lenguaje natural
Generalización	Nula ante lo no previsto	Parcial: reconoce variantes de lo ya visto
Fallo típico	Falso positivo por falta de contexto	Afirmación segura y equivocada

Ninguno de los dos enfoques sustituye por completo al otro: las reglas son buenas detectando y malas decidiendo qué importa, y el modelo es débil detectando por sí solo pero fuerte explicando y priorizando lo que ya fue detectado. El diseño que este trabajo adopta usa las reglas como fuente —de ahí que Wazuh se conserve como origen de las alertas— y el modelo como capa de triaje sobre ellas, nunca al revés.

2.9. Políticas de decisión determinista y separación entre analizar y actuar

Un motor de triaje que, además de clasificar una alerta, decidiera con el mismo componente generativo qué acción de respuesta ejecutar, heredaría todos los riesgos de la sección anterior en el punto de mayor consecuencia del sistema: el momento de actuar sobre la infraestructura del cliente. La alternativa que sostiene el diseño de este trabajo separa esa decisión en dos capas con motivos de cambio distintos: una *política* determinista, expresada como una tabla indexada por la clase de la alerta y el contexto del activo, que *propone* la acción de menor impacto capaz de resolver la amenaza; y un *perfil de cliente* configurable, que *filtra* esa propuesta según lo que el negocio del cliente concreto permite, degradándola a una de menor impacto o reteniéndola para validación humana. El modelo de lenguaje interviene únicamente para redactar la justificación de una decisión ya tomada por la política, nunca para tomarla. Esta separación es, en sí misma, una aplicación directa del principio de auditabilidad: un error se puede atribuir a la clasificación, a la política o al perfil, y se sabe cuál de los tres corregir, algo que no sería posible si toda la decisión residiera dentro de un único componente generativo.

Esta separación es también la que permite incorporar el agente descrito en la sección 2.6 sin renunciar a la auditabilidad. El agente no sustituye a ninguna de las tres capas: actúa dentro de la tercera, eligiendo entre las acciones que el catálogo ofrece y que el perfil ya ha autorizado, cuando la contención en el equipo afectado resulta insuficiente. La frontera se mantiene, por tanto, en el mismo sitio: la política determina *qué* debe ocurrir y el modelo, en su papel de agente, propone *cómo* lograrlo dentro de lo permitido, sujeto al mismo filtro del perfil del cliente: automático lo que este permite, validación humana lo que retiene.

2.10. Métricas de evaluación de un clasificador binario

Evaluar si el triaje asistido mejora sobre el método de referencia exige medir su desempeño como clasificador binario —amenaza o no amenaza— sobre una matriz de confusión con cuatro categorías: verdadero positivo (VP), cuando el sistema marca como amenaza una alerta que efectivamente lo es; falso positivo (FP), cuando la marca como amenaza sin serlo; verdadero negativo (VN), cuando correctamente no la marca; y falso negativo (FN), cuando una amenaza real pasa desapercibida. De esa matriz se derivan las métricas de la tabla 2.2.

Tabla 2.2. Métricas de clasificación empleadas en la evaluación

Métrica	**Fórmula**	**Qué mide**
Precisión (P)	$\frac{VP}{VP+FP}$	De lo marcado como amenaza, cuánto lo era realmente
Exhaustividad (R)	$\frac{VP}{VP+FN}$	De las amenazas reales, cuántas se detectaron
Medida F_1	$\frac{2\,P\,R}{P+R}$	Equilibrio entre precisión y exhaustividad
Tasa de falsos positivos	$\frac{FP}{FP+VN}$	El ruido operativo que sufre el analista

De estas cuatro, la tasa de falsos positivos es la que responde más directamente al problema planteado en la sección 2.1: mide exactamente la proporción de ruido que un analista tendría que descartar manualmente. La exhaustividad, por su parte, es la métrica que no puede sacrificarse bajo ninguna circunstancia: un sistema que reduce el ruido a costa de dejar pasar amenazas reales no es una mejora, sino un riesgo nuevo.

2.11. Entornos de red virtualizados para experimentación en seguridad

Ejercitar y medir un motor de triaje exige un entorno donde generar actividad legítima y hostil sobre una red con vulnerabilidades conocidas, sin exponer una infraestructura productiva. Las plataformas de este tipo se dividen en dos familias con una diferencia determinante para este trabajo: las que *simulan* un dispositivo —reproducen su comportamiento observable sin ejecutar su sistema operativo real— y las que lo *virtualizan* o lo *emulan*, ejecutando una imagen real del sistema dentro de un contenedor o una máquina virtual. Solo la segunda familia expone servicios genuinos que una herramienta de escaneo de vulnerabilidades pueda encontrar, lo que descarta a los simuladores puramente didácticos como plataforma de laboratorio para este proyecto.

Containerlab es una de las plataformas de esa segunda familia: define la topología de red como un fichero de texto versionable, orquesta contenedores y máquinas virtuales ligeras según ese fichero, y crea automáticamente una red de gestión fuera de banda que conecta todos los nodos entre sí [5]. Su reproducibilidad —una topología descrita en un fichero puede destruirse y recrearse de forma idéntica— es lo que hace posible que las condiciones de una campaña de evaluación puedan repetirse de manera fiel, un requisito directo de la reproducibilidad exigida a la propia evaluación del capítulo 3.

CAPÍTULO 3

METODOLOGÍA

El trabajo se organizó en seis fases sucesivas, correspondientes una a una con los objetivos específicos enunciados en la Introducción, más una séptima fase de consolidación documental. Cada fase produjo un entregable concreto sobre el que se apoya la siguiente, y las decisiones de diseño que no podían anticiparse al inicio —porque dependían de una medición, no de un criterio— se tomaron y registraron en el momento en que la medición estuvo disponible, con su fecha. Esta sección documenta esas decisiones en el orden en que se tomaron.

3.1. Enfoque general: modelar en lugar de esperar

La primera decisión metodológica, y la que condiciona a todas las demás, se tomó antes de escribir ninguna línea de código: al no disponer de acceso a un cliente real ni a su sistema propietario de análisis de alertas, el trabajo optó por *modelar* un cliente genérico en lugar de *analizar* uno concreto. Analizar un cliente real habría producido un prototipo ajustado a ese cliente; modelar uno genérico obligó a separar explícitamente, desde el principio, lo que cualquier empresa comparte —una infraestructura de red organizada por capas, con una superficie de gestión remota presente en todas ellas— de lo que varía de una a otra —su catálogo de acciones permitidas, su política de continuidad de servicio, la criticidad de sus activos—. Esa separación es la que permite que el prototipo resultante sea configurable por cliente sin cambios de código, y es la misma lógica que, más adelante, resolvió la ausencia de una fuente real de alertas: en vez de esperar a que un sistema propietario estuviera disponible, se sustituyó por una plataforma de detección basada en reglas —Wazuh— desplegada dentro de un laboratorio propio, con la ventaja adicional de que su nivel de regla podía servir directamente como método tradicional de referencia.

3.2. Fase 1: modelado del cliente y del caso de uso

Se construyó un modelo de cliente genérico describiendo su infraestructura de red por capas —punto de entrega, equipo de borde, conmutación, zona desmilitarizada, servidores

internos, puestos de trabajo, dispositivos IoT y plano de gestión—, identificando en cada una qué expone hacia la administración remota y qué ocurre si se interrumpe. De ese modelo se derivaron cinco puntos de variabilidad configurables entre clientes —el esquema de la alerta, el repertorio de acciones y su canal, la política de continuidad, el inventario y criticidad de activos, y la severidad de referencia— y un conjunto de restricciones operativas que no son negociables en ningún cliente: que ningún servicio se detenga sin autorización, que no exista una ventana de mantenimiento sobre la que apoyarse, que toda acción sea reversible porque no hay manos físicas junto al equipo, y que ninguna acción pueda cortar el propio canal de gestión por el que se administra y se responde.

Sobre ese modelo se acotó el caso de uso a una sola familia de alertas —el acceso no autorizado a servicios de gestión expuestos—, deliberadamente definida por el tipo de patrón y no por el equipo concreto que lo sufre: la misma fuerza bruta contra un servicio de administración se reconoce igual en un conmutador que en un servidor de ficheros, lo que permite que el sistema generalice al parque de equipos de cualquier cliente en lugar de aprender las particularidades de uno solo.

3.3. Fase 2: estado del arte y registro de requisitos

Se revisaron, por un lado, las plataformas comerciales de detección y respuesta extendida y gestionada y sus limitaciones documentadas, y por otro, las aplicaciones de modelos de lenguaje a la seguridad, incluyendo sus riesgos específicos. De ese análisis se derivó un registro único de veinte requisitos funcionales y catorce no funcionales, cada uno trazado hasta la limitación de mercado o el criterio de diseño que lo motiva. Cuando, más adelante, el encuadre del proyecto se desplazó del segmento de pequeña y mediana empresa hacia la red de un cliente modelado en la Fase 1, ese registro se revisó íntegramente contra el nuevo contexto: ningún requisito original dejó de aplicar, seis cambiaron de matiz y siete se añadieron —principalmente los que más adelante formalizarían la política de continuidad de servicio—, lo que se documentó como una revisión explícita y no como una reescritura silenciosa.

3.4. Fase 3: construcción del entorno de pruebas

El entorno de pruebas se construyó como una red de cliente emulada con Containerlab, reproduciendo desde el punto de entrega del proveedor hacia dentro: un equipo de borde, un conmutador de red interna, un puesto de usuario, un dispositivo de tipo IoT con servicios expuestos y un servidor con vulnerabilidades documentadas —Metasploitable2, con diecinueve puertos abiertos y fallos conocidos y públicos—, todos conectados además a una red de gestión fuera de banda por la que operan, sin contaminar el tráfico observado, el motor de triaje y un

nodo auditor con Nmap (figura 3.1).

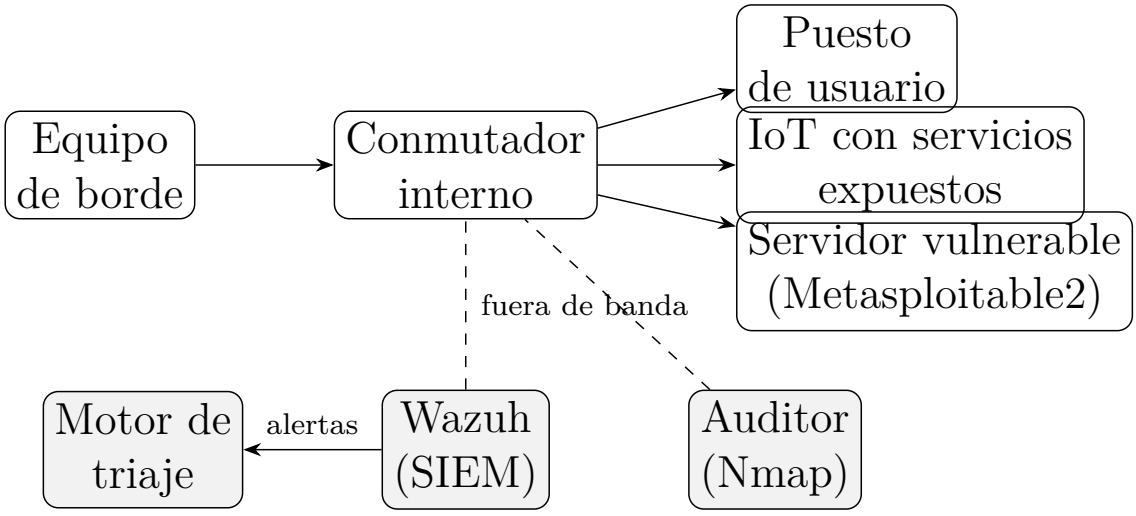


Figura 3.1. Topología del laboratorio (Containerlab): la red del cliente (plano de datos) y, fuera de banda, el plano de gestión con Wazuh, el motor de triaje y el auditor.

Un mismo laboratorio debía resolver dos necesidades que, se verificó explícitamente, no son intercambiables: un *ground truth* de vulnerabilidades —qué falla exactamente en cada nodo— y un *ground truth* de alertas —si una alerta concreta es un verdadero o un falso positivo—. El primero sale directamente de la composición documentada de la topología; el segundo exigió un puente explícito: cada alerta que Wazuh generaba sobre la actividad del laboratorio se contrastaba contra el inventario de vulnerabilidades del nodo al que apuntaba, y de esa comparación —con revisión manual, no completamente automatizable— salía la etiqueta de verdadero o falso positivo, conservando en todo momento el nivel de regla original de Wazuh como el dato que más adelante serviría de baseline.

El resultado, cerrado el 31 de agosto de 2026 y ampliado el 11 de septiembre con una segunda y una tercera familia de ataque (secciones 4.11 y 4.11.1), fue un conjunto de 410 alertas reales —466 tras la segunda y tercera familia, y 600 tras la campaña de casos difíciles de la Fase 6 (sección 3.7)— generadas sobre el propio laboratorio, etiquetadas y particionadas por campaña: 20 verdaderos positivos, 16 falsos positivos y 374 alertas de ruido de plataforma —telemetría de evaluación de configuración que no corresponde a ningún patrón del caso de uso acotado—, todas ellas concentradas, las soportadas, en un único activo y servicio (acceso por SSH al servidor con vulnerabilidades documentadas). Esta concentración en una sola familia de ataque, verificada al construir el conjunto y no descubierta después, es la que más adelante bloquearía el ajuste fino del clasificador y limitaría el alcance estadístico de la evaluación del capítulo 4: se documenta aquí como una condición conocida del método, no como un hallazgo posterior.

3.5. Fase 4: diseño de la arquitectura

El diseño se apoyó en tres puntos de aislamiento deliberados, cada uno resuelto mediante un adaptador que esconde una dependencia externa hoy provisional: un módulo de ingesta que aísla la fuente de alertas concreta —hoy Wazuh, mañana el sistema propietario del cliente—; una interfaz de análisis de dos operaciones, `clasificar` y `justificar`, que aísla el modelo de lenguaje concreto detrás de cada una; y un conector que aísla el protocolo de comunicación hacia el equipo objetivo —hoy SSH sobre un catálogo cerrado de acciones—. Ninguno de los tres exige reescribir la lógica de decisión del motor de triaje cuando la pieza que ocultan cambie (figura 3.2).

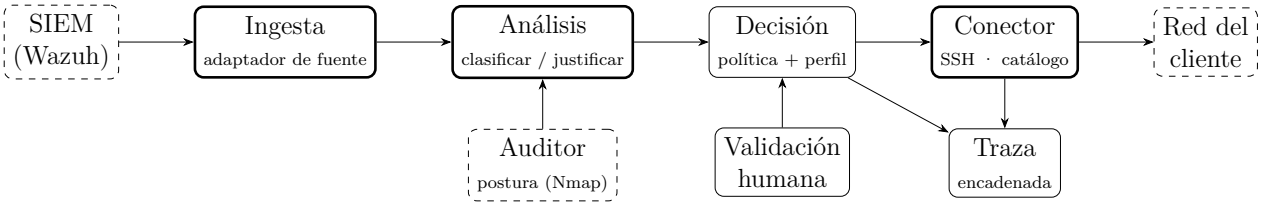


Figura 3.2. Arquitectura del motor de triaje y sus tres puntos de aislamiento (trazo grueso: ingesta, análisis y conector). La fuente de alertas, el modelo de lenguaje y el canal hacia el objetivo quedan tras un adaptador cada uno, sin tocar la lógica de decisión.

Sobre esa interfaz de análisis se definieron dos perfiles de despliegue según el hardware disponible: un perfil híbrido para el equipo de desarrollo entonces disponible —un procesador de propósito general de ocho núcleos, sin GPU dedicada y con dieciséis gigabytes de memoria—, que combina un codificador ligero para clasificar con un modelo generativo pequeño para justificar en línea; y un perfil de un único componente generativo especializado, viable únicamente con una GPU dedicada o memoria sustancialmente mayor. La decisión inicial del 24 de agosto de 2026 estimaba, sobre mediciones publicadas en hardware comparable, que un modelo generativo de tres mil ochocientos millones de parámetros cuantizado rendiría entre diez y doce *tokens* por segundo en el equipo disponible. La medición directa realizada el 2 de septiembre de 2026 dio un resultado muy distinto —aproximadamente 3,7 *tokens* por segundo—, lo que se documentó de inmediato como una corrección y no se ocultó: con ese rendimiento, un modelo de ese tamaño quedaba fuera del presupuesto de latencia tolerable para una validación humana interactiva, y el justificador se implementó finalmente sobre un modelo generativo de mil millones de parámetros. La interfaz de análisis, precisamente por estar aislada del modelo concreto, no tuvo que rediseñarse para absorber ese cambio.

También en la Fase 4 se resolvió que la decisión de qué acción proponer nunca recae en el modelo de lenguaje, por las razones expuestas en la sección 2.9 del marco teórico: una tabla de política determinista, indexada por la clase de la alerta, propone la acción de menor impacto que resuelve la amenaza, y un perfil de cliente configurable la filtra —permitiéndola,

degradándola a una de menor impacto o reteniéndola para validación humana— según lo que ese cliente concreto permite. Se definió además el plan de métricas de la evaluación —precisión, exhaustividad, medida F_1, tasa de falsos positivos, acierto de prioridad y dos métricas de continuidad específicas de este caso de uso: acciones disruptivas indebidas y retención correcta—, y se fijó como método tradicional de referencia el nivel de regla de Wazuh, por ser literalmente el enfoque basado en reglas y firmas contra el que el proyecto se planteó comparar.

3.6. Fase 5: implementación

El prototipo se implementó en Python 3, empleando únicamente su biblioteca estándar más `pyyaml` para la configuración, sin gestor de paquetes ni entorno virtual adicional para el motor de decisión, y con `unittest` como marco de pruebas. La implementación se dividió en tres subproyectos:

- **Núcleo de decisión.** La cadena ingesta → análisis → política → perfil → traza, ejecutable en lote sobre el conjunto de alertas etiquetado, con un clasificador de referencia determinista —una regla fija sobre la postura de exposición del activo, sin aprendizaje— detrás de la interfaz `clasificar`. Este clasificador se mantiene como una pieza provisional deliberada: el ajuste fino de un codificador quedó bloqueado precisamente por la concentración en una sola familia de ataque documentada en la Fase 3, sin variedad suficiente de clases para entrenar y validar un modelo que generalizara.
- **Lazo en vivo.** Un conector que traduce cada acción abstracta del catálogo cerrado a un comando concreto sobre el nodo objetivo por SSH, con verificación de idempotencia previa a la ejecución, validación de los parámetros contra caracteres de control de shell, y una segunda verificación posterior para confirmar el efecto; una validación humana por terminal que muestra la alerta, la clase, la prioridad, la justificación y la acción propuesta antes de ejecutarla; y el registro de traza auditable de cada decisión. El ejecutor SSH real se inyecta como una función sustituible, de forma que la lógica de idempotencia y de verificación se pudo probar con un ejecutor simulado sin necesidad del laboratorio activo. Al cierre del trabajo, la traza pasó a escribirse **encadenada por resumen criptográfico**: cada registro lleva el resumen SHA-256 del registro anterior y el suyo propio, calculado sobre su contenido en forma canónica, de modo que alterar un campo, borrar, insertar o reordenar cualquier registro rompe la cadena a partir de ese punto y un verificador señala en cuál; el servicio en tiempo real retoma la cadena del fichero al reiniciarse, para que varias sesiones formen una sola. Lo que la cadena sola no cubre —el truncado del fichero por el final, porque lo que queda

sigue encadenado— lo cubre un **ancla**: tras cada registro, el servicio envía el resumen y el número de registros como un evento de *syslog* al propio Wazuh, que lo conserva en su fichero de alertas bajo una regla local; Wazuh es otro sistema, ya es la fuente de verdad del triaje, y su fichero no lo controla quien controla la traza. El verificador contrasta la traza con las anclas y distingue tres fallos: truncada, rehecha —el fichero borrado y creado de nuevo con el mismo nombre, que se reconoce por un identificador de linaje derivado del primer registro— y alterada. Los tres se reprodujeron en vivo. Una consecuencia se detectó y se corrigió en el mismo ensayo: las anclas vuelven al servicio como alertas, y sin descartarlas en la ingesta cada ancla generaba una decisión, y cada decisión un ancla. En el mismo cierre, el conector dejó de usar la credencial por defecto del nodo vulnerable —una contraseña en el código y sin verificar la identidad del equipo remoto— y pasó a un usuario dedicado con clave, verificación del equipo contra una lista propia, y **privilegios en el nodo restringidos a un fichero de *sudo* generado desde el propio catálogo de acciones**, de modo que la frontera de lo que el conector puede ejecutar y la frontera de lo que el motor puede ordenar son el mismo dato. El aprovisionamiento termina comprobando que un comando del catálogo se ejecuta y que dos fuera de él —uno con otro programa y otro con el mismo programa y otros argumentos— se deniegan, y el lazo completo se verificó en vivo con esa credencial, igual que su rotación: la clave nueva se añade en todos los nodos, se verifica en todos, y solo entonces se retira la anterior, con el cambio anotado en el SIEM; si un nodo falla, nada cambia. La reversión de una acción (RF-18) pasó también de promesa del catálogo a operación: se deriva del registro de la decisión y del catálogo, la ejecuta el mismo conector, se verifica que el estado desapareció y se anota en la misma cadena de la traza. Todo ello, junto con la configuración por cliente, el arranque, la verificación periódica y la conducta ante fallos, quedó recogido en una guía de operación para la empresa, escrita sobre un ensayo completo del modo en tiempo real y no sobre lo que el código debería hacer.

- **Justificador con modelo de lenguaje real.** Un modelo generativo de mil millones de parámetros, cuantizado a cuatro bits, invocado mediante un proceso independiente a un motor de inferencia en un entorno Conda separado del resto del prototipo. Cada justificación se somete a una verificación de anclaje automática —ninguna dirección IP mencionada puede ser distinta de la de la alerta, y el texto debe referenciar al menos un campo concreto de esta— y, si no la supera o el modelo falla, se degrada a una plantilla determinista que por construcción sí ancla, de forma que ninguna alerta queda nunca sin justificación.

Sobre esa misma interfaz se añadió un cuarto componente, no previsto en el diseño original de la Fase 4 sino incorporado tras detectar en la Fase 6 que el justificador de mil millones de

parámetros, aunque anclaba correctamente sus afirmaciones, las describía con un significado equivocado: un módulo de generación aumentada por recuperación que indexa, mediante un modelo de *embeddings* —inicialmente el propio modelo generativo, y después uno dedicado, por las razones que el capítulo 4 documenta—, un corpus curado de fichas sobre las técnicas MITRE ATT&CK, las reglas de Wazuh, las vulnerabilidades del laboratorio pertinentes al caso de uso y, tras la evaluación, los motivos por los que una alerta se descarta; y recupera los tres pasajes más afines a cada alerta —por similitud coseno, según la ecuación (2.1) del marco teórico (sección 2.5)— antes de invocar al modelo generativo. Tres decisiones de diseño acotan deliberadamente su alcance. La consulta de recuperación se construye únicamente a partir de campos estructurados de la alerta —la técnica, la familia y el servicio— y de la clase que el motor ya decidió, nunca del registro completo del evento, por la misma razón de seguridad que ya impedía tratar ese campo como instrucción. El corpus se mantiene reducido y curado a mano, en vez de indexar documentación completa, porque el objetivo era verificar si corregir el conocimiento del modelo mejoraba la corrección de su justificación, no maximizar la cobertura de la recuperación. Y el corpus se divide por construcción en dos familias según la pregunta que cada ficha responde —por qué una amenaza importa, o por qué una alerta no lo es—, de modo que la clase decidida determina también en qué subconjunto se busca.

Cerrada la evaluación, la implementación se extendió con dos piezas que convierten el procesamiento por lotes en una operación continua. La primera es un **servicio en tiempo real** que consume las alertas según la fuente las produce, las agrupa en incidentes dentro de una ventana temporal configurable, y presenta cada incidente al analista para su validación sin esperar al final de una campaña; admite tres modos de operación —justificación por plantilla determinista, justificación con el modelo de lenguaje y recuperación aumentada, o delegación en el agente de mitigación— que permiten intercambiar latencia por riqueza de la explicación sin alterar la decisión, que es determinista en los tres casos. La segunda es un **agente de mitigación** que, ante un incidente cuya contención en el propio equipo no basta, escala hacia el siguiente nodo con capacidad de contención —del anfitrión al cortafuegos— con el filtro del perfil aplicado a cada salto: automática la contención local sobre un atacante externo, con validación humana el salto a un cortafuegos perimetral. Al cierre del trabajo ese filtro rige tanto la escalada determinista como la del agente —que contiene primero en el equipo atacado y solo escala al cortafuegos cuando ese paso falla—, y se convirtió en el comportamiento por defecto del servicio (sección 4.8.4).

El origen de esta segunda pieza conviene declararlo, porque no procede de los objetivos del plan sino de un requisito planteado por el **tutor industrial**: en una red en producción el equipo atacado puede estar caído, comprometido o sin canal de administración, y el sistema no puede quedarse por ello sin capacidad de respuesta. La necesidad es, por tanto, operativa y está avalada por la empresa; lo que quedó abierto —y se resuelve en el capítulo de resultados—

es qué mecanismo la implementa mejor, si una regla determinista sobre la topología o un agente que razone la estrategia. Para poder ejercitar y medir ambas alternativas se añadió al laboratorio una topología con un equipo de borde administrable y un banco de escenarios de ataque organizados por el comportamiento que deben provocar en el motor.

Ambas piezas comparten una propiedad de diseño que conviene destacar: **la decisión y la contención son deterministas e instantáneas en cualquier modo**, y el modelo de lenguaje solo interviene en la explicación y, en el caso del agente, en la elección de la estrategia de escalada. El coste de inferencia afecta, por tanto, a la calidad de la justificación, no al tiempo de respuesta ante la amenaza.

La capa de invocación del modelo se rehízo al detectarse que la anterior estaba defectuosa. El prototipo lanzaba un proceso del motor de inferencia por cada llamada, pasándole un parámetro de temperatura que ese ejecutable no admite y que, en consecuencia, acababa incorporado al propio texto que el modelo leía; además, al no aplicarse la plantilla de conversación del modelo, la salida devolvía el enunciado repetido antes de la respuesta. El justificador toleraba ambos defectos porque su enunciado termina en un marcador por el que la salida se recorta, y ese recorte descartaba el sobrante; el agente, cuyo enunciado carece de ese marcador, nunca llegaba a producir una acción interpretable. La invocación pasó a realizarse contra un **servicio de inferencia residente**, lo que resuelve las tres cosas a la vez —la plantilla la aplica el servicio, la temperatura viaja como un campo estructurado y no puede filtrarse al enunciado, y el modelo permanece cargado entre llamadas— y habilita además la generación restringida por esquema descrita en el marco teórico. Cada petición desactiva además la caché de prefijos del servicio, porque se midió que con ella la salida a temperatura cero dependía de lo que el servicio hubiera procesado antes, lo que incumplía el requisito de reproducibilidad. La identificación de versión del justificador se incrementó con cada uno de estos cambios, de modo que las campañas anteriores siguen atribuidas a la invocación con la que se midieron.

Todo el prototipo se validó con una batería de pruebas unitarias que, en el momento de escribir este informe, asciende a 957 casos en el motor de triaje —más 63 del marco de evaluación, 39 de la tubería de construcción del conjunto de datos, 137 del laboratorio del banco y 187 del visor web (React/TypeScript)—, ejecutados con generadores, ejecutores y capa de red simulados que permiten probar la lógica de decisión, de idempotencia, de escalada y de degradación sin necesidad de un modelo de lenguaje cargado ni de un laboratorio en ejecución.

3.7. Fase 6: evaluación

La evaluación se diseñó como una campaña reproducible por línea de comandos que procesa una partición del conjunto de alertas etiquetado, calcula la clasificación del prototipo y, sobre la misma partición, calcula el mejor punto de un barrido del nivel de regla de Wazuh como umbral de clasificación —el método tradicional de referencia—, de forma que ambos se comparan siempre bajo las mismas condiciones y contra el mismo *ground truth*. Para la calidad de la justificación se definieron dos evaluaciones complementarias: una automática, que mide la tasa de anclaje —el porcentaje de justificaciones cuyas afirmaciones referencian campos que existen realmente en la alerta—, y una manual, sobre una muestra, en la que un revisor humano juzga si el contenido de la explicación es correcto y no solo si cita los campos adecuados. Esta distinción resultó decisiva: es precisamente la que permitió detectar que un anclaje perfecto no garantiza una justificación semánticamente correcta, y la que motivó incorporar la recuperación aumentada descrita en la sección anterior.

Para acotar el límite de la señal de origen, se añadió una cuarta campaña de *casos difíciles* cuya verdad no provenía de la heurística de etiquetado sino del propio experimento: ataques diseñados para que el método de referencia fallara —fuerza bruta lanzada desde la misma dirección declarada como administrador legítimo, y ráfagas cuyo nivel de regla no supera ningún umbral—, con dos réplicas para dar variedad. Esta campaña llevó el conjunto etiquetado a sus 600 alertas (212 soportadas), con la partición de evaluación de 300 que usa el capítulo de resultados. Sobre ella se entrenó un árbol de decisión (CART), no como clasificador de producción sino como *instrumento de validación*: para ver qué reglas descubriría el aprendizaje supervisado a esta escala con una verdad independiente de la heurística. El árbol redescubrió las reglas ya codificadas y aportó una nueva —que la ráfaga pesa más que el origen declarado—, que se adoptó como código auditable; el detalle está en el capítulo 4.

Para aislar el efecto de esa incorporación, la campaña se ejecutó dos veces sobre el mismo subconjunto de dieciocho alertas soportadas de la partición de evaluación, con el modelo generativo y la temperatura de muestreo fijadas en ambos casos: una vez con el justificador simple, y otra con la recuperación aumentada activada, comparando en cada caso el mismo par de alertas para poder atribuir cualquier diferencia observada a la sola presencia del módulo de recuperación.

CAPÍTULO 4

RESULTADOS Y DISCUSIÓN

Este capítulo presenta los resultados de la campaña de evaluación de la Fase 6, ejecutada sobre la partición de evaluación del conjunto de alertas etiquetado —205 alertas, de las cuales 18 pertenecen al caso de uso acotado y tienen, por tanto, una clase verdadera contra la que se puede medir—, con el perfil de cliente empresarial y el justificador de mil millones de parámetros a temperatura de muestreo nula. La sección 4.11 amplía después esa partición con una segunda familia de ataque generada en el mismo laboratorio, y comprueba que el resultado principal se sostiene sobre ella.

4.1. Resultado principal: clasificación frente al método de referencia

La pregunta que la fase debía responder es si el triaje asistido aporta algo sobre el nivel de regla de Wazuh tomado como método tradicional de referencia. La tabla 4.1 resume el resultado.

Tabla 4.1. Prototipo frente al baseline de Wazuh (partición de evaluación, $n = 205$)

Métrica	**Prototipo**	**Baseline (nivel ≥ 5)**
Precisión	1,000	0,154
Exhaustividad	1,000	1,000
Medida F_1	1,000	0,267
Tasa de falsos positivos	0,000	0,282

La matriz de confusión del prototipo es $VP = 10$, $FP = 0$, $VN = 195$, $FN = 0$; la del baseline en su umbral óptimo es $VP = 10$, $FP = 55$, $VN = 140$, $FN = 0$. Ambos métodos detectan la totalidad de las amenazas reales de la partición —exhaustividad de 1,0 en los dos casos—, pero el prototipo suprime por completo el ruido: el baseline deja pasar 55 falsos positivos y el prototipo, ninguno. Los 195 negativos del prototipo son los 187 eventos de ruido de plataforma que clasifica como no soportados más las 8 alertas del administrador legítimo

que reconoce como actividad legítima, dos categorías que el nivel de regla no sabe distinguir. Esta supresión del ruido operativo sin pérdida de exhaustividad es, precisamente, el resultado que el proyecto se propuso demostrar.

4.2. La progresión de la medida, y por qué se reporta completa

La primera medición de esta fase no dio 1,000 sino **0,556** de precisión, con una matriz de $VP = 10$, $FP = 8$, $VN = 187$. Ambos números son reales y describen dos configuraciones distintas del mismo motor; se reportan los dos porque la diferencia entre ellos es, en sí misma, el hallazgo más instructivo de la evaluación.

Las 18 alertas soportadas de la partición corresponden todas al mismo activo y al mismo servicio —acceso por SSH al servidor con vulnerabilidades documentadas del laboratorio—, de las cuales diez son intentos de acceso reales y ocho son actividad de un administrador legítimo, todas desde la misma dirección de origen. La señal que separa unas de otras no es la postura de exposición del activo, porque el servicio SSH está igualmente expuesto en los dos casos: es la *legitimidad del origen*. En la primera medición el clasificador no disponía de esa señal —aunque el etiquetado del conjunto de datos sí la había usado—, así que marcaba las dieciocho como amenaza potencial y acumulaba ocho falsos positivos.

La corrección no consistió en entrenar nada, sino en declarar esa señal como configuración del cliente: el perfil enumera los orígenes legítimos, el enriquecimiento marca cada alerta en consecuencia y el clasificador produce una categoría propia de actividad administrativa legítima para esos casos. Con esa configuración los ocho falsos positivos pasan a negativos correctos y la precisión alcanza 1,000.

Quedan dos límites que conviene no suavizar. El primero es que se trata de una señal determinista basada en la dirección de origen, que es suplantable: en un despliegue real la legitimidad debería apoyarse en una autenticación más fuerte y no solo en la dirección. El segundo es que el clasificador cubre así cuatro de las seis categorías del caso de uso; las dos restantes —el acceso ya consumado y la exposición indebida de un servicio de gestión— exigen señales que el conjunto de datos disponible no contiene, y son las que motivan el clasificador con ajuste fino que permanece bloqueado por la concentración de ese conjunto en una sola familia de ataque.

4.3. Por qué el óptimo del baseline es débil

El nivel de regla de Wazuh se evaluó como clasificador barriendo su umbral de decisión; la tabla 4.2 resume la medida F_1 resultante en cada punto.

Tabla 4.2. Medida F_1 del baseline según el umbral de nivel de regla

nivel $\geq$	3	4	5	6	7	8	9	10	11	12
F_1	0,093	0,267	0,267	0,035	0,035	0,167	0,167	0,167	—	—

El mejor punto del baseline —$F_1 = 0,267$, en los umbrales 4 y 5— no se acerca en ningún caso al resultado del prototipo, ni siquiera al de su primera medición ($F_1 = 0,714$ antes de la configuración de orígenes legítimos, $F_1 = 1,000$ después). La curva, además, no es monótona: subir el umbral no mejora de forma limpia, porque los verdaderos positivos y buena parte del ruido conviven en los mismos niveles de severidad. Esto confirma que el nivel de regla, por sí solo, no basta como criterio de prioridad, y justifica la reclasificación que realiza el motor de triaje.

4.4. Agrupación por incidente: menos elementos que revisar

Un analista no debería revisar dieciocho alertas casi idénticas, sino los incidentes que representan. El motor colapsa las ráfagas que comparten origen, activo, servicio y familia dentro de una ventana temporal, por encima de la correlación que la propia fuente de alertas ya realiza. La tabla 4.3 recoge el efecto medido.

Tabla 4.3. Reducción de elementos a revisar por agrupación en incidentes

Entrada	**Elementos**	**Tras agrupar**
18 alertas soportadas	18	2 incidentes (9×)
205 alertas (partición completa)	205	4 incidentes ($\approx$ 51×)

Los dos incidentes soportados son exactamente el ataque y la actividad del administrador. Triar sus dos representantes con el motor produce las mismas decisiones correctas que triar las dieciocho alertas por separado, con nueve veces menos trabajo. El matiz honesto es que ambos incidentes pertenecen a la misma familia, porque el conjunto de datos es de una sola: sobre tráfico más variado la agrupación distinguiría más incidentes y el factor de reducción sería distinto.

4.5. Operación y continuidad del servicio

El tiempo de clasificar y decidir sobre una alerta, con el clasificador determinista, fue del orden de $0,01$ ms por alerta, muy por debajo del presupuesto de latencia establecido.

Justificar con el modelo de lenguaje de mil millones de parámetros tomó, en esta campaña, aproximadamente 36 s por alerta —más que en una invocación aislada, porque cada justificación recarga el modelo de 808 MB en un proceso nuevo; en un despliegue real el modelo permanecería cargado entre invocaciones—. La cobertura del clasificador —la proporción de alertas que caen dentro del caso de uso acotado— fue de 0,088: solo el 8,8 % de las alertas de un Wazuh recién desplegado corresponden a un patrón clasificable, y el resto es telemetría interna de la propia plataforma. La tasa de escalado a validación humana fue de 0,0 sobre este conjunto, porque el clasificador determinista nunca reportó una confianza por debajo del umbral de escalado en estas alertas. Una ampliación posterior, recogida al final de esta sección, eleva esa cifra al ejercitar el escenario más exigente de continuidad.

Sobre la métrica de continuidad —el requisito que distingue a este trabajo de un sistema de respuesta automatizada convencional—, no se registró ninguna acción disruptiva indebida: ninguna acción propuesta a partir de una alerta que resultó ser un falso positivo alcanzaba un servicio que el cliente prestara. El escenario más exigente de esta métrica —una acción de alto impacto que deba ser efectivamente retenida por la validación humana— no llegó a ejercitarse sobre este conjunto de datos, lo que se documenta como una limitación de la muestra y no como una garantía general.

4.5.1. Ampliación posterior: continuidad bajo un atacante interno

El escenario más exigente que el conjunto anterior no pudo ejercitar sí se ejercitó al cierre del trabajo, con la **conciencia de impacto** (requisitos RF-17 y RF-19): para cada contención, el sistema determina desde el inventario del perfil a quién bloquea y qué servicios detiene, y **retiene para validación humana todo bloqueo que recaiga sobre un activo interno** del cliente; cortar el canal de gestión es un veto duro. Se midió sobre la partición de evaluación de 300 alertas con el perfil empresarial corregido para describir la red real del laboratorio, donde el atacante es *interno*: las ráfagas de fuerza bruta salen del puesto de un empleado.

El coste de esa política es medible. Las contenciones automáticas —ejecutadas sin intervención humana— pasaron de 69 a **cero**: las 69 bloqueaban el puesto de un empleado, y **dos de ellas eran falsos positivos** —un empleado desconectado sin que nadie lo aprobara— que el indicador de continuidad original no registraba, porque solo miraba las acciones que alcanzan un servicio prestado al exterior. Una métrica nueva, la de bloqueos automáticos indebidos, cubre ese punto ciego: esas dos pasan a **cero**. La tasa de escalado a validación humana subió en consecuencia de 0,067 a **0,297** —89 de 300—, sin que la clasificación cambiara (recall 1,000, tasa de falsos positivos 0,009). La automatización queda para los orígenes externos, y lo propio pasa siempre por una persona.

El mecanismo es configurable por perfil. Un perfil de cliente **bancario** —continuidad

extrema, con activos críticos que no pueden interrumpirse— lo lleva al límite, y se demostró en vivo sobre un laboratorio de banco de **catorce nodos** con servicios, dependencias reales y cascada entre activos.

Esta ampliación es posterior a la campaña principal de la Fase 6 y confirma, sobre el escenario que antes quedaba fuera de la muestra, la propiedad que distingue a este trabajo: un triaje que acierta más sin cortar lo que el cliente presta.

4.6. Calidad de la justificación: anclada, pero inicialmente poco fiable

La verificación automática de anclaje —que ninguna dirección IP mencionada sea distinta de la de la alerta y que el texto referencie al menos un campo concreto de ella— dio una tasa de 18 sobre 18: el cien por ciento de las justificaciones del modelo de mil millones de parámetros pasó la verificación. Sin embargo, la revisión manual de esa misma muestra reveló que un anclaje perfecto no garantiza una explicación correcta. Un ejemplo textual generado por el modelo antes de incorporar recuperación aumentada:

> *«La regla 5760 es un protocolo de seguridad para la autenticación de usuarios en redes. El MITRE T1110.001 es un conjunto de técnicas de seguridad para la autenticación de usuarios...»*

La regla 5760 de Wazuh no es un protocolo: es la regla que detecta un fallo de autenticación por SSH. La técnica T1110.001 no es un conjunto de técnicas de seguridad: es *Password Guessing*, una técnica de ataque del catálogo MITRE ATT&CK. El modelo cita los campos correctos —de ahí que ancle— pero describe su significado al revés, y de las dieciocho justificaciones generadas solo cuatro textos resultaron distintos entre sí, porque una temperatura de muestreo nula sobre un único nodo produce salidas casi idénticas. La conclusión de esta medición es metodológicamente importante: la tasa de anclaje mide que el modelo no inventa campos, no que acierte su significado, y no debe leerse como un indicador de calidad de la justificación.

4.7. El contraste con recuperación aumentada

Sobre las mismas dieciocho alertas, con el mismo modelo y la misma temperatura, se repitió la campaña con el módulo de recuperación aumentada activado. La tasa de anclaje automática no cambió —se mantuvo en 18 sobre 18, porque mide otra cosa—, pero la corrección semántica de la justificación, evaluada manualmente, sí lo hizo. La misma alerta que antes producía el texto citado en la sección anterior, generó con recuperación aumentada:

«La regla 5760 de Wazuh indica un ataque de fuerza bruta SSH en curso desde una IP.»

correcta y respaldada por los pasajes recuperados del corpus curado sobre las reglas de fuerza bruta SSH. En una alerta de falso positivo, el contraste fue igual de marcado: de «la regla 5710 define el protocolo SSH como un protocolo seguro» —una afirmación sin sentido— a «la regla 5710 de Wazuh indica que se probó un nombre de usuario que no existe por SSH, un indicio de ataque de fuerza bruta», que es la lectura correcta del evento.

Tres matices, medidos y no solo observados, acompañan esta mejora. Primero, la diversidad de textos no aumentó —siguieron saliendo solo cuatro textos distintos de dieciocho—, porque la causa de esa baja diversidad es el propio conjunto de datos concentrado en un nodo, no el mecanismo de justificación. Segundo, la recuperación resultó imprecisa en el identificador exacto de la regla: para una alerta de la regla 5760, el sistema recuperó el grupo completo de reglas de fuerza bruta SSH en lugar del identificador preciso; aun así, el dominio recuperado era el correcto, y por eso la justificación mejoró. Tercero, con recuperación aumentada el modelo tiende a parafrasear los pasajes recuperados más que a razonar de forma original sobre la alerta concreta, lo que es preferible a inventar una explicación pero sigue estando lejos de la redacción de un analista.

4.8. Revisión de la invocación y medición con un modelo mayor

Al preparar la ejecución sobre una máquina con unidad de procesamiento gráfico se detectó que la capa de invocación del modelo era defectuosa, en los términos descritos en el capítulo 3. El hallazgo tiene valor metodológico por encima del técnico: el justificador *parecía* funcionar porque el recorte de su salida descartaba el sobrante, de modo que el defecto quedaba enmascarado precisamente en el componente que sí se medía, mientras inutilizaba por completo el que no se había podido ejercitar. Un componente puede pasar su batería de pruebas y estar inservible en ejecución si nadie recorre la ruta de llamadas real desde el punto de entrada que usaría una persona.

Corregida la invocación, se repitió la medición con un modelo de ocho mil millones de parámetros servido de forma residente. La tabla 4.4 recoge el contraste frente a la medición original.

Tabla 4.4. Justificación con el modelo pequeño y con uno mayor, misma partición

	Modelo de 1B	**Modelo de 8B**
Anclaje sin recuperación	18/18	18/18
Anclaje con recuperación	18/18	18/18
Degradaciones a plantilla	0	0
Textos distintos (de 18)	4	3 sin RAG · 5 con RAG
Duración de la campaña	10 min 46 s	51 s

La mejora operativa es sustancial —la campaña completa pasa de casi once minutos a menos de uno, porque el modelo deja de recargarse en cada invocación— y la calidad de la explicación sube de forma apreciable. Pero el resultado más instructivo no está en la tabla.

4.8.1. La recuperación mejora los verdaderos positivos y empeora los falsos

Comparando las justificaciones de la misma alerta con y sin recuperación, con el modelo mayor, el efecto se separa con nitidez según la clase de la alerta.

Sobre un **verdadero positivo**, la recuperación hace exactamente lo que se esperaba de ella: la explicación pasa de una descripción genérica del servicio —«el servicio SSH, que es un protocolo de acceso remoto comúnmente utilizado, está expuesto y puede ser vulnerable a ataques»— a una lectura precisa del hecho: «indica un ataque de fuerza bruta SSH en curso desde la IP 192.168.1.10».

Sobre un **falso positivo**, en cambio, la degrada. La alerta procede de una dirección que el perfil del cliente declara como administración legítima, el motor la clasifica correctamente como actividad legítima, y esa clase viaja dentro del enunciado que recibe el modelo. Aun así, la justificación con recuperación afirma: «indica un ataque de fuerza bruta SSH en curso desde la IP 192.168.1.1». El conocimiento recuperado —fichas sobre reglas de fuerza bruta— se impone sobre la clasificación, y el modelo narra el pasaje en lugar de explicar la decisión.

Las consecuencias son dos, y ninguna es menor. La primera es operativa: el analista recibe una pantalla en la que el motor dice «actividad legítima» y la justificación dice «ataque en curso». Una contradicción interna erosiona la confianza en el sistema más deprisa que un error franco. La segunda es metodológica: **ambas justificaciones superan la verificación de anclaje**, porque las dos citan la dirección y el activo correctos. La métrica de anclaje mide que el modelo no inventa campos; no mide que respete la decisión que se le ha dado ni que su lectura sea coherente con ella.

4.8.2. La corrección, y el intercambio que revela

El origen del defecto resultó estar en el enunciado, no en la recuperación: se pedía al modelo que explicara «por qué esta alerta importa» *también* cuando el motor acababa de decidir que no importa. Sobre esa pregunta, y con fichas sobre técnicas de ataque delante, la respuesta coherente para el modelo era describir un ataque. Se corrigió haciendo que la pregunta dependa de la clase ya decidida —para una clase sin amenaza se pide explicar *por qué no lo es*—, encuadrando los pasajes como descripción de la técnica que hizo saltar la regla y no de lo ocurrido, y añadiendo a los datos el motivo real por el que la alerta es un falso positivo: que el origen figura como administración declarada por el cliente.

La contradicción desaparece. Sobre la misma alerta, la justificación pasa a ser: «la alerta no es una amenaza debido a que el origen 192.168.1.1 es declarado como administración legítima por el cliente». Pero la corrección tiene un coste medible, recogido en la tabla 4.5.

Tabla 4.5. Los tres estados de la justificación con recuperación, misma partición y mismo modelo

	Enunciado ciego a la clase	**Enunciado por clase**	**Enunciado, consulta y corpus por clase**
Justificaciones ancladas	18/18	11/18	18/18
de las amenazas reales	10/10	10/10	10/10
de los falsos positivos	8/8	1/8	8/8
Contradicen la clasificación	8	0	0
Descartes en plantilla	0	7	0

Las dos primeras columnas corresponden a la corrección del enunciado; la tercera, a la sección 4.8.3, se lee después.

Siete de los ocho falsos positivos dejan de anclar y caen a la plantilla determinista. La razón es que, al recibir el motivo ya formulado, el modelo lo enuncia sin necesidad de citar ningún campo concreto —«el origen de la actividad es declarada como administración legítima»— y la verificación, con buen criterio, lo rechaza. Pedir explícitamente en el enunciado que mencione la dirección y el activo recuperó solo uno de los siete.

El intercambio merece leerse con cuidado, porque es contraintuitivo: **la tasa de anclaje bajó de 1,00 a 0,61 mientras la calidad subía**. Antes, los ocho falsos positivos producían un texto fluido, anclado y *equivocado*; ahora producen, en su mayoría, una descripción factual generada por plantilla, que es correcta por construcción. Para un sistema que un analista debe poder creer, la segunda situación es preferible sin discusión.

Es también la demostración más limpia de la tesis que atraviesa este capítulo: **la tasa de anclaje no es una medida de calidad**. Una intervención que mejora las justificaciones la empeora, y una que las empeorase podría mejorarla. Reportarla sola, sin la lectura del contenido, induce a error.

Esto refuerza, desde un ángulo nuevo, la conclusión de la sección anterior: el anclaje es condición necesaria y claramente insuficiente. Durante estas mediciones se detectó además una segunda forma de superarlo sin aportar nada —un modelo que repite el enunciado en lugar de responder produce un eco que contiene todos los campos de la alerta y, por tanto, ancla—, que obligó a tratar esa repetición como un fallo de generación. Una métrica automática de explicabilidad debe defenderse de las maneras de satisfacerla sin cumplir su propósito.

4.8.3. Cerrar el circuito: recuperar el motivo del descarte

Que siete de los ocho descartes cayeran a la plantilla no era un límite del modelo, sino del material que se le daba. El corpus contenía únicamente fichas sobre técnicas de ataque y contramedidas: no había una sola que explicara por qué algo *no* es una amenaza. Y aunque la hubiera habido, no se habría recuperado, porque la consulta al índice se construía siempre con la técnica MITRE y la palabra «contramedida», es decir, con sabor a ataque, también para una alerta que el motor acababa de descartar.

La corrección aplicó a la recuperación el mismo principio que ya se había aplicado al enunciado: **la clase ya decidida gobierna la pregunta, y la pregunta gobierna qué conocimiento hace falta**. Se materializó en tres capas. Se añadieron al corpus tres fichas de descarte, una por cada clase sin amenaza —origen declarado como administración legítima, servicio no expuesto según el inventario, alerta fuera del perímetro del caso de uso—. La consulta pasó a recibir la clase, y para una clase sin amenaza busca el motivo del descarte en lugar de la técnica. Y el corpus se partió en dos familias que no se mezclan: una amenaza no ve fichas de descarte, y un descarte no ve fichas de ataque; las fichas sobre la postura del activo, pertinentes para las dos preguntas, quedan en ambas. Esta separación se impone por construcción y no se confía a la distancia entre vectores, porque se comprobó que, aun con la consulta de descarte, la segunda y la tercera ficha recuperadas seguían siendo material de ataque.

El resultado es la tercera columna de la tabla 4.5: las dieciocho justificaciones vuelven a generarse y ninguna contradice al motor. Las dieciocho se leyeron una a una. Los ocho descartes citan el motivo real —«la alerta no es una amenaza debido a que el origen de la alerta (192.168.1.1) figura entre las direcciones de administración declaradas por el cliente en su perfil, lo que indica que el intento de acceso es realizado por un administrador que ha errado la credencial, en lugar de un atacante»— y las diez amenazas citan origen, activo

y servicio expuesto. La clasificación no cambió en ningún estado, como corresponde a un componente en el que el modelo no participa.

La tabla, leída de izquierda a derecha, es la formulación más precisa de la tesis de este capítulo: **el 1,00 de la primera columna y el 1,00 de la tercera son el mismo número significando cosas opuestas**. El primero medía ocho textos anclados y falsos; el tercero, dieciocho textos anclados y correctos. Entre ambos, el 0,61 fue una mejora. Ninguna lectura de la tasa, por sí sola, distingue las tres situaciones; solo el contenido lo hace, y por eso la evaluación registra ahora el anclaje desglosado por clase y la consulta que se usó en cada recuperación.

La lectura dejó además una forma nueva de narrar material de referencia como hecho, más sutil que la contradicción: en una corrida intermedia, un verdadero positivo se explicaba «a través de la técnica MITRE T1021.004», que no es la técnica de la alerta sino la de un pasaje recuperado sobre un servicio vecino. La verificación de anclaje se extendió con el mismo criterio que ya aplicaba a las direcciones: todo identificador de técnica que aparezca en el texto debe ser uno de los de la alerta, o la técnica padre de uno de ellos. Sobre la corrida vigente la guarda no altera ninguna de las dieciocho justificaciones, porque ninguna cita identificadores; se incorpora igualmente, porque con recuperación los pasajes traen por construcción identificadores de técnicas vecinas, y la tentación existe aunque en esta corrida no se haya materializado.

4.8.4. El agente de mitigación, medido en condiciones válidas

Corregida la invocación, el agente descrito en la sección 2.6 pudo por fin ejercitarse de verdad. Los resultados llegan en tres escalones, y ninguno es el que cabría esperar.

Con el modelo pequeño y sin restringir la salida, el agente **nunca** produjo una acción interpretable: agotaba los pasos y terminaba sin contener nada. Al restringir la salida con el esquema derivado del catálogo, el comportamiento cambió de golpe: el modelo emitió al primer intento un objeto sintácticamente impecable y conforme al esquema. Pero declaraba la mitigación *sin haber invocado ninguna herramienta*: ninguna acción ejecutada, ninguna dirección bloqueada. La restricción había resuelto el problema del formato y dejado intacto el del contenido. Fue esa observación la que obligó a que el resultado de una mitigación se lea del registro de lo ejecutado y nunca de lo que el modelo afirma.

Con el modelo de ocho mil millones de parámetros y la salida restringida, sobre tres ejecuciones del escenario de escalada, el agente **no completó el ciclo por sí mismo en ninguna** de ellas, aunque el esquema sí redujo el coste a la tercera parte —de nueve a tres segundos por incidente— al evitar pasos malgastados en salidas inservibles. En las tres, sin

embargo, **la amenaza quedó contenida**, porque la escalada determinista actúa cuando el agente no concluye.

La lectura honesta es que **ni siquiera un modelo de ese tamaño sostiene un ciclo de razonamiento y acción fiable en este dominio** con el hardware y el montaje evaluados. Es un resultado negativo y va a contracorriente de la tendencia actual a incorporar agentes a todo, lo que lo hace más digno de reportarse que de omitirse. Conviene además situarlo: la capacidad que el tutor industrial pidió —no quedarse sin contención cuando el equipo atacado no responde— está entregada y funciona siempre, porque no depende del modelo. Lo que no está demostrado es que delegarla en un agente aporte algo sobre la regla determinista en un perímetro de dos dispositivos, y esa es exactamente la hipótesis que la sección 2.6 anticipaba como no verificable a esta escala.

Esa conclusión tuvo una consecuencia de diseño que se aplicó al cierre del trabajo: la escalada determinista dejó de ser solo el respaldo del agente y pasó a ser **el camino por defecto** del servicio. Cuando la contención en el equipo afectado no se ejecuta o no se verifica, el lazo recorre la cadena de contención hacia el perímetro sin necesidad del modo agente. Cablearla exigió una pieza que faltaba: cada salto pasa ahora por el filtro del perfil de cliente, de modo que un veto duro —cortar el plano de gestión, una acción sin reversión— salta el dispositivo, una degradación sustituye la acción, y la validación humana se pide cuando el perfil la exige para ese impacto, y no en todos los casos. Sin esa pieza, la escalada habría podido ejecutar en el cortafuegos algo que el perfil prohíbe. Se verificó en vivo sobre la topología con equipo de borde administrable, con la credencial de mínimo privilegio: detenido el demonio SSH del equipo atacado tras un ataque real, el paso en el anfitrión falló (código 255), el servicio pidió aprobación para el cortafuegos —impacto que alcanza el servicio, humano siempre en el perfil empresarial—, aplicó y verificó la regla de filtrado en el equipo de borde, dejó la decisión encadenada en la traza con la orden efectiva, y la reversión posterior, derivada de esa misma traza, eliminó la regla y quedó anotada en la cadena. Cinco segundos entre la alerta y la contención verificada en el perímetro.

4.9. La recuperación: un techo, su causa y su desplazamiento

La intuición razonable ante una recuperación imprecisa es ampliar el corpus. Se midió esa hipótesis y resultó falsa para este montaje. Al pasar de 29 a 39 fichas —añadiendo técnicas del catálogo MITRE ATT&CK semánticamente próximas a las ya presentes— la calidad de la recuperación se degradó a la mitad, según recoge la tabla 4.6.

Tabla 4.6. Efecto de ampliar el corpus sobre la calidad de la recuperación

Métrica	**29 fichas**	**39 fichas**
Aciertos en la primera posición	0,25	0,00
Rango recíproco medio	0,41	0,19

La causa es el modelo de *embeddings* empleado, que es el mismo modelo generativo de mil millones de parámetros: no discrimina entre técnicas de ataque semánticamente vecinas —adivinar contraseñas frente a descifrarlas, por ejemplo—, de modo que más fichas equivalen a más confusión. Se revirtió la ampliación y se conservaron las 29 fichas originales. La lección práctica es que, con un modelo de este tamaño, el corpus no puede crecer de forma útil: ampliarlo exige antes un modelo de *embeddings* dedicado. Es un resultado negativo, y se reporta como tal porque delimita con precisión dónde está el techo del montaje actual.

Un segundo resultado, medido sobre el mismo banco de doce alertas sintéticas repartidas entre las cuatro familias del perímetro, apunta al mismo componente. La consulta al corpus puede construirse de dos maneras: fija, componiéndola con los campos estructurados de la alerta, o *agéntica*, dejando que el propio modelo formule qué conviene buscar. Esta segunda se diseñó de forma que *añade* su aportación a la consulta fija, con la intención expresa de que nunca pudiera ser peor. La medición dice lo contrario, según recoge la tabla 4.7.

Tabla 4.7. Calidad de la recuperación según cómo se formule la consulta

Métrica	**Consulta fija**	**Consulta agéntica**
Aciertos en la primera posición	0,33	0,25
Rango recíproco medio	0,47	0,32

La explicación que se dio entonces fue que añadir texto a una consulta semántica no añade señal sino que la desplaza: la consulta es un único vector, y las palabras que el modelo agrega lo mueven. Es una explicación plausible, y resultó ser incompleta, como muestra la subsección siguiente.

4.9.1. Con un modelo de *embeddings* dedicado

Los dos resultados anteriores señalaban al mismo componente, así que se sustituyó. El modelo generativo dejó de hacer de *embedder* y se incorporó *bge-m3*, un modelo de *embeddings* multilingüe de 1 024 dimensiones, con la agrupación por CLS con la que fue entrenado —detalle que no es menor: un modelo generativo usado como *embedder* exige la media, y cambiar de

modelo sin cambiar la agrupación degrada la recuperación en silencio—. La tabla 4.8 recoge el efecto sobre el mismo banco, la misma consulta fija y el mismo corpus.

Tabla 4.8. Calidad de la recuperación según el modelo de *embeddings*, consulta fija

Métrica	**Generativo de 1B**	**bge-m3**
Aciertos en la primera posición	0,33	0,50
Aciertos entre las tres primeras	0,58	0,83
Aciertos entre las cinco primeras	0,75	1,00
Rango recíproco medio	0,47	0,70

La ficha correcta pasa a estar siempre entre las cinco primeras. Y el cambio corrige el hallazgo anterior sobre la consulta agéntica: con el *embedder* dedicado, la consulta que el modelo amplía recupera algo mejor que la fija —rango recíproco medio de 0,73 frente a 0,70, aciertos entre las tres primeras de 0,92 frente a 0,83—. Que fuera peor no era una propiedad de la técnica, como se había interpretado, sino un artefacto del *embedder* débil, y la intención de diseño —que aumentar la consulta nunca la empeore— se sostiene. El informe conserva la explicación anterior tal como se dio, porque ilustra un riesgo real de la evaluación: una explicación mecanicista convincente de un resultado negativo puede estar describiendo el instrumento y no el fenómeno.

El techo que la tabla 4.6 había fijado también se desplaza. Repetida la ampliación del corpus con técnicas hermanas del perímetro —ocho, extraídas del catálogo oficial, de 32 a 40 fichas— y medida con *bge-m3*, el rango recíproco medio baja de 0,70 a 0,66, una sola alerta pierde la primera posición frente a su técnica hermana, y los aciertos entre las tres y las cinco primeras no cambian. Donde el modelo pequeño partía la métrica por la mitad, el dedicado paga cuatro centésimas. Crecer el corpus pasa de inviable a viable, que es exactamente lo que la recomendación derivada del resultado negativo decía que ocurriría. La ampliación no se incorporó al corpus de referencia porque ninguna alerta del perímetro la necesita todavía.

4.9.2. Reproducibilidad de la generación (RNF-03)

Al repetir el banco dos veces sin cambiar código ni corpus, la consulta agéntica dio rangos recíprocos medios de 0,77 y 0,81, mientras la fija daba 0,70 las dos veces. Se descartó en cadena cada fuente posible de variación: el generador devolvía el mismo texto para el mismo enunciado cinco veces seguidas, las doce consultas coincidían en dos pasadas dentro del mismo proceso y entre dos procesos distintos, y el *embedder* devolvía vectores idénticos bit a bit. Lo único que quedaba era lo que el servidor había procesado *antes*: tras ejecutar una campaña completa, cuatro de las doce consultas cambiaron para los mismos enunciados.

La causa es la caché de prefijos del servidor de inferencia, que reutiliza cómputo de peticiones anteriores y altera el camino numérico de la generación, de modo que a temperatura cero la salida depende del estado del servidor. Desactivarla en cada petición —al coste de recalcular enunciados de unas decenas de *tokens*— hizo que el banco diera cifras idénticas antes y después de una campaña. Las cifras reproducibles son las que este capítulo reporta; las dos anteriores no deben citarse. La lección excede a este montaje: **la reproducibilidad de un modelo de lenguaje no la garantiza el muestreo determinista por sí solo, sino la infraestructura que lo sirve**, y un requisito no funcional de reproducibilidad debe verificarse contra esa infraestructura y no darse por cumplido al fijar la temperatura.

El mismo fenómeno reapareció en el otro componente numérico al servir el modelo de *embeddings* de forma residente, por la misma razón de tiempo que el generador —cargarlo en cada llamada costaba unos dos segundos por vector; residente, cien milisegundos—. Al comparar los vectores del servidor con los del subproceso, el mismo texto daba vectores distintos según qué otros textos viajaran en el mismo lote, en los dos caminos, con una similitud coseno consigo mismo de 0,99975; y esa diferencia bastó para voltear un empate del banco de recuperación. La regla que lo resuelve es la misma: un texto por llamada, siempre, de modo que el vector sea función del texto y de nada más. Con ella, subproceso y servidor coinciden hasta la última cifra, el índice construido por uno vale para el otro, y el banco reproduce las cifras publicadas.

De paso quedó a la vista un segundo defecto, menor: el modelo mayor antepone a veces un preámbulo —«la búsqueda que te recomiendo es:»— y sitúa la consulta en la línea siguiente. La limpieza de la salida se quedaba con la primera línea no vacía, que era el preámbulo, y lo pegaba a la consulta. Es el mismo tipo de defecto que el de la invocación (sección 4.8): un componente que funcionaba con un modelo deja de hacerlo con otro sin que ninguna prueba lo detecte, porque las pruebas fijaban la salida del modelo.

4.10. Verificación sobre el conjunto completo

Para descartar que el resultado dependiera de la partición elegida, la clasificación —sin el justificador, por su coste de tiempo— se repitió sobre las 410 alertas del conjunto completo (36 soportadas). Las tasas obtenidas —precisión 1,000, exhaustividad 1,000, F_1 1,000 y tasa de falsos positivos 0,000, con matriz $VP = 20$, $FP = 0$, $VN = 390$, $FN = 0$— resultaron idénticas a las de la partición de evaluación, lo que era el resultado esperado: el clasificador es determinista y no se entrena, así que no existe diferencia entre entrenar y evaluar sobre él. La partición se respetó de todas formas por rigor metodológico.

4.11. Segunda familia de ataque: reconocimiento

La limitación central de la evaluación anterior era la concentración del conjunto de datos en una sola familia de ataque. Para empezar a levantarla se generó, en el mismo laboratorio y con la misma tubería de etiquetado, una segunda familia: el reconocimiento de servicios. Se eligió porque el adaptador de Wazuh ya la reconocía —las reglas 5706, «no se recibió cadena de identificación», y 5701, «identificación de protocolo incorrecta», ambas del demonio SSH y del grupo *recon*— y porque admite el mismo par de escenarios que la familia original: un nodo de la red escaneando al objetivo (amenaza), el administrador declarado comprobando que un servicio responde (falso positivo) y el auditor del propio servicio escaneando por programa (actividad propia).

El primer hallazgo fue del propio laboratorio. La campaña original ya incluía un escaneo desde el auditor y no había producido una sola alerta; la causa se entendió ahora: un escaneo SYN no completa el establecimiento de la conexión y el demonio SSH nunca lo ve, y un escaneo de conexión completa con *nmap* tampoco deja rastro, porque cierra con un reinicio de conexión y el demonio de 2007 del objetivo solo registra el cierre ordenado. Las campañas se construyeron con sondeos de conexión y cierre ordenado —seis barridos de puertos y tres sondeos de protocolo contra el puerto 22 desde el nodo atacante, cuatro comprobaciones desde el administrador y dos barridos desde el auditor—, congelando únicamente la ventana temporal de cada campaña. Dos campañas idénticas en composición, una por partición, aportaron quince alertas cada una: nueve del atacante, cuatro del administrador y dos del auditor. El conjunto pasó de 410 a 440 alertas, y la partición de evaluación, de 18 a 31 alertas soportadas de dos familias.

Tabla 4.9. Clasificación sobre la partición ampliada ($n = 220$, 31 soportadas), antes y después de declarar al auditor como origen legítimo

Métrica	**Auditor no declarado**	**Auditor declarado**	**Baseline**
Precisión	0,905	1,000	0,237
Exhaustividad	1,000	1,000	1,000
Medida F_1	0,950	1,000	0,384
Tasa de falsos positivos	0,010	0,000	0,303
Falsos positivos (de 220)	2	0	61

El resultado de clasificación, en la tabla 4.9, trajo el segundo hallazgo. Los diecinueve verdaderos positivos —diez de fuerza bruta y nueve de reconocimiento— se detectan todos, y los cuatro sondeos del administrador declarado se descartan; pero aparecen dos falsos positivos, y son exactamente los dos barridos del auditor. **El prototipo habría bloqueado**

a su propio instrumento de medida. No lo había hecho antes solo porque los escaneos del auditor no dejaban rastro. La corrección no tocó el código: la dirección del auditor se declaró como origen legítimo en el perfil de cliente, que es el lugar previsto para ello (RNF-14), y se fijó en la topología del laboratorio para que sea determinista. Con esa declaración la clasificación vuelve a precisión 1,000 y tasa de falsos positivos 0,000, con matriz $VP = 19$, $FP = 0$, $VN = 201$, $FN = 0$; el baseline queda en 0,237 de precisión y 0,303 de falsos positivos, peor que sobre la familia original porque el nivel 8 de la regla 5701 le impide subir el umbral. Sobre el conjunto completo de 440 alertas (62 soportadas, $VP = 38$) las tasas del prototipo son idénticas.

El tercer hallazgo está en la justificación, y es el más instructivo. Las treinta y una justificaciones se generaron y anclaron; leídas, las del reconocimiento eran incorrectas: un sondeo de puerto se explicaba como «intentando explotar la vulnerabilidad del servicio SSH», y una de ellas situaba al activo «en la dirección 192.168.1.10», que es la del atacante. La causa era doble. El enunciado no incluía la familia de la alerta —la clase, *vp_intento_acceso*, es común a todas las familias por diseño, así que el modelo no tenía forma de saber que se trataba de un escaneo— y las etiquetas MITRE con que Wazuh marca esas dos reglas, T1021.004 (servicios remotos: SSH) y T1190 (explotación de aplicación expuesta), apuntan a acceso y explotación, no a descubrimiento. Se añadieron al enunciado la familia y los papeles explícitos de cada dirección («origen de la actividad», «activo afectado»), y la ficha del corpus que describe la familia incorporó el vocabulario observable de Wazuh. Con ello, la regla 5706 pasa a explicarse como descubrimiento de servicios expuestos y la confusión de papeles desaparece; pero la regla 5701 sigue narrándose como «T1190, explotación de aplicación expuesta», porque eso dice la etiqueta de la fuente y el enunciado ordena citar los datos. **La explicación hereda las etiquetas de la fuente**, y ninguna instrucción al modelo lo cambia sin cambiar antes lo que el motor le dice. Es el argumento medido a favor de una clase propia para el reconocimiento —con la misma acción, pero su propio nombre, su propia prioridad y su propia explicación—, que se deja como recomendación porque altera la capa de decisión.

El cuarto hallazgo afecta al banco de recuperación de la sección 4.9: sus alertas sintéticas de reconocimiento llevaban las etiquetas MITRE «ideales» (T1046, T1595), no las que Wazuh pone de verdad, así que el banco no reproducía los datos que el sistema recibe. Se añadieron dos casos con las etiquetas reales. Con la ficha de la familia enriquecida, esos dos casos recuperan la ficha correcta en primera y segunda posición, a un coste de cuatro centésimas de rango recíproco medio sobre los doce casos originales; sobre los catorce, la consulta fija da 0,67 y la agéntica 0,77, con la ficha correcta entre las cinco primeras en el 93 % y el 100 % de los casos respectivamente.

4.11.1. Tercera familia: el servicio en claro, y el nivel de regla pierde amenazas

Con el mismo método se generó una tercera familia, el servicio expuesto en claro: conexiones al telnet del objetivo desde el nodo atacante (ocho, en dos minutos), desde el administrador declarado (tres) y desde el auditor (dos), en dos campañas idénticas. Dos piezas de integración hicieron falta y las dos son hallazgos sobre la fuente, no sobre el prototipo. El demonio de telnet escribe en un fichero distinto del de autenticación, así que el reenvío hubo de ampliarse; y el decodificador de fábrica de Wazuh no extrae la dirección de origen del formato con que ese demonio registra la conexión —«connect from 192.168.1.10 (192.168.1.10)», con el nombre entre paréntesis al final—, de modo que la alerta llegaba sin origen, y **sin origen no hay a quién bloquear ni a quién descartar**. Se resolvió donde corresponde, en la configuración del SIEM, con un decodificador local que el laboratorio instala al arrancar el gestor.

El conjunto pasó a 466 alertas y la partición de evaluación a 42 soportadas de tres familias. La clasificación del prototipo no cambió: precisión 1,000, exhaustividad 1,000, tasa de falsos positivos 0,000, con matriz $VP = 27$, $FP = 0$, $VN = 206$, $FN = 0$. Lo que cambió fue el baseline, y en la dirección que la tabla 4.2 anticipaba: **por primera vez el nivel de regla dejó pasar amenazas**. Cada conexión al servicio en claro genera una alerta de nivel 3, por debajo del umbral óptimo del baseline (≥ 5); solo la correlación de Wazuh por repetición —seis conexiones en dos minutos, nivel 10— supera el umbral. De las ocho amenazas de esta familia en la partición el baseline detecta una, y su exhaustividad sobre las tres familias cae a 0,741 ($VP = 20$, $FP = 61$, $VN = 145$, $FN = 7$), con precisión 0,247 y tasa de falsos positivos 0,296. Bajar el umbral para recuperarlas le costaría muchos más falsos positivos: un único nivel numérico no puede separar a la vez el ruido de plataforma, que es alto, de una conexión en claro, que es baja. El prototipo no usa el nivel: usa la familia, la postura del activo y el origen, y por eso detecta las ocho sin añadir un falso positivo. Las 42 justificaciones se generaron y anclaron; las de telnet describen la conexión a un servicio en claro expuesto, con la ficha de la familia recuperada, y no contradicen al motor. El banco de recuperación incorporó dos casos con las etiquetas reales —una de las reglas llega sin técnica MITRE, y la correlación llega etiquetada como fuerza bruta— y queda en 16 casos: consulta fija 0,64 y agéntica 0,83 de rango recíproco medio, con la ficha correcta entre las cinco primeras en el 94 % y el 100 % de los casos.

4.12. El clasificador entrenado, y lo único que aportó

El clasificador con ajuste fino previsto en el diseño quedó bloqueado durante todo el trabajo por la concentración del conjunto de datos, y la ampliación a tres familias no lo desbloqueaba por una razón más profunda que el tamaño: **las etiquetas las generaban las**

mismas señales que usa el clasificador determinista. El etiquetado marcaba como falso positivo lo que venía de un origen declarado y como verdadero positivo lo que iba contra un servicio expuesto; las cinco reglas hacen exactamente eso; así que su 1,000 era una tautología y cualquier modelo entrenado sobre esos datos solo podía aprender a copiar las reglas. Había además un confundido: en las campañas el administrador fallaba con un usuario inexistente (reglas 5710 y 5712 de Wazuh) y el atacante con *root* (5760 y 5763), de modo que la propia regla de Wazuh delataba el origen.

Se generó por eso una cuarta campaña, en dos réplicas, diseñada para que las reglas fallaran, con la verdad declarada por el experimento y no por la heurística —quién lanzó qué, desde dónde, con qué intención—: una fuerza bruta con *root* **desde la dirección declarada del administrador** (las reglas dicen falso positivo; es una amenaza, por suplantación o por equipo comprometido); un error de contraseña seguido de un acceso correcto desde un puesto no declarado (las reglas dicen amenaza; es un usuario legítimo que se equivocó); y sus dos simétricos, que rompen el confundido. Sobre la partición resultante —300 alertas, 106 soportadas— las cinco reglas cayeron a exhaustividad 0,770 con 20 falsos negativos, todos la fuerza bruta desde la dirección del administrador, y el baseline de Wazuh, que ignora el origen, las superó en exhaustividad (0,920). La señal de origen, que hasta aquí solo había aportado, mostró su otra cara.

Con esos datos se entrenó un árbol de decisión —CART sobre Gini, sin dependencias, porque a la escala de cien filas un modelo grande memoriza y no puede validarse— con rasgos estructurados: los que ya usa el determinista, la regla y el nivel de Wazuh, y tres rasgos de *ráfaga* calculados sobre el flujo de alertas del mismo origen (cuántas en un minuto, cuántas reglas distintas, si hubo un acceso correcto después). La dirección de origen no es un rasgo: memorizaría direcciones, no comportamientos. Entrenado sobre la partición de entrenamiento y medido sobre la de evaluación, el árbol alcanzó precisión 1,000 y exhaustividad 0,989, corrigiendo los veinte falsos negativos y los dos falsos positivos. Y lo que aprendió cabe en cuatro líneas: si hay más de ocho alertas del mismo origen en un minuto, es amenaza, venga de donde venga; si no, un origen declarado es falso positivo, y una conexión al servicio en claro desde un origen no declarado es amenaza. **El árbol redescubrió las reglas escritas a mano y añadió una**, la de la ráfaga, que fue la raíz del árbol con cualquier profundidad y cualquier tamaño mínimo de hoja, y que apareció idéntica al intercambiar las particiones. Sin los rasgos de ráfaga, el árbol se aferraba a las reglas de Wazuh —el confundido— y empeoraba.

La consecuencia se aplicó al motor, no como modelo sino como **sexta regla determinista**: una ráfaga por encima del umbral del perfil es amenaza aunque el origen esté declarado, con la confianza reducida a 0,6 para que la contención pase por validación humana antes de bloquear al administrador. La única aportación del aprendizaje supervisado con evidencia fuerte se adoptó como una regla auditable; la otra hoja del árbol —el error seguido de acceso, con tres

casos y etiquetas mezcladas— no tenía evidencia para cambiar el diseño, y se dejó. Con la sexta regla, sobre la misma partición, el prototipo recupera exhaustividad 1,000 con precisión 0,978 y tasa de falsos positivos 0,009 —frente a 0,310 del baseline de Wazuh en su umbral óptimo sobre esta misma partición de 300 alertas—; los dos falsos positivos que quedan son el error de tecleo, que el diseño trata como intento de acceso y que aquí se cuenta contra él. Se verificó en vivo: una fuerza bruta desde la dirección del administrador produce una amenaza retenida para validación humana, con la ráfaga en la justificación, y la justificación generada la nombra como la razón que pesa más que la procedencia. Sobre el conjunto completo de 600 alertas (212 soportadas) las cifras son las mismas ($VP = 174$, $FP = 4$, $VN = 422$, $FN = 0$).

La lectura para el trabajo futuro es distinta de la que el diseño anticipaba. El aprendizaje supervisado no aportó aquí un componente en tiempo de ejecución sino una *herramienta de descubrimiento de reglas*: con datos cuya verdad no venga de la heurística, un modelo pequeño e interpretable dice qué regla falta, y esa regla se adopta como código auditable. El codificador con ajuste fino sigue teniendo sentido solo cuando el conjunto de datos tenga la variedad que hoy no tiene, y su primer competidor será este árbol.

4.13. Limitaciones y análisis de errores

Cinco limitaciones, todas medidas y no solo declaradas, acotan el alcance de estos resultados. Primero, el tamaño de la muestra soportada —dieciocho alertas de una sola familia en la evaluación original, cuarenta y dos de tres familias tras las ampliaciones de las secciones 4.11 y 4.11.1, todas sobre un único activo— hace que los porcentajes se sostengan sobre pocos casos, y que la correlación de orden de prioridad no pudiera calcularse por falta de variedad; es la limitación central y estructural de toda la evaluación, y las dos familias añadidas la reducen sin eliminarla. Segundo, la separación entre administrador y atacante descansa en una señal determinista basada en la dirección de origen, que es suplantable, y cubre cuatro de las seis categorías del caso de uso. Tercero, el clasificador con ajuste fino permanece bloqueado por la misma concentración del conjunto de datos, así que el desajuste esperado entre un modelo preentrenado sobre prosa y una entrada de eventos estructurados queda documentado, pero no cuantificado; lo que sí se cuantificó, con un árbol sobre datos cuya verdad no venía de la heurística, es que el aprendizaje supervisado aporta a esta escala una sola regla, la de la ráfaga, que se adoptó (sección 4.12). Cuarto, la calidad de la justificación ya no está limitada por el hardware —un modelo de ocho mil millones de parámetros responde en menos de medio segundo sobre una tarjeta gráfica de escritorio, sección 4.8— sino por tres cosas distintas: que el conocimiento recuperado puede imponerse sobre la clasificación y contradecirla, que la recuperación no puede mejorarse ampliando el corpus mientras se use el mismo modelo generativo como *embedder* (sección 4.9), y que de las dos cuantizaciones públicas disponibles

del modelo especializado en seguridad seleccionado en el diseño, ninguna resultó utilizable: ambas declaran plantillas de conversación que no corresponden a su arquitectura, y producen repetición del enunciado o fuga de marcas de control. La medición con modelo mayor debió hacerse, por tanto, con un modelo generalista de referencia, lo que aísla el efecto del tamaño pero deja sin comprobar el de la especialización. Quinto, el auditor de vulnerabilidades no es infalible: la ausencia de un hallazgo no demuestra que un activo sea seguro, y la postura de exposición usada como contexto de decisión se tomó de los hallazgos reales de una única campaña de escaneo.

El resultado global, no obstante, es positivo y medido sobre el problema que el proyecto se propuso atacar: el triaje asistido suprime el ruido operativo sin sacrificar exhaustividad y sin comprometer la continuidad del servicio, y sus límites —la cobertura parcial de categorías, la fragilidad de la señal de origen y la capacidad del modelo de justificación— tienen todos una causa identificada y un camino de mejora conocido que la propia interfaz de análisis del prototipo ya permite incorporar sin rediseñar el sistema.

Conclusiones y recomendaciones

El objetivo general de este trabajo era desarrollar un prototipo que clasificara y priorizara alertas de seguridad, justificando cada decisión con un modelo de lenguaje apoyado en generación aumentada por recuperación ejecutada en local, y que redujera los falsos positivos sin sacrificar la detección de amenazas reales. Los resultados del capítulo 4 sostienen que ese objetivo se cumplió dentro del caso de uso acotado sobre el que se evaluó: el prototipo suprimió los falsos positivos que el nivel de regla de Wazuh deja pasar —tasa de 0,000 frente a 0,282 en el caso acotado original de una familia, y de 0,009 frente a 0,310 sobre la partición final de 300 alertas—, sin dejar de detectar ninguna amenaza real y sin ejecutar ninguna acción que hubiera interrumpido indebidamente un servicio del cliente. Una ampliación posterior —la conciencia de impacto, medida sobre el laboratorio donde el atacante es interno— ejercitó el escenario más exigente de esta métrica: al retener para validación humana todo bloqueo que recae sobre un activo propio, las contenciones automáticas indebidas —dos que el indicador de continuidad original no veía— pasan a cero y el escalado a validación humana sube al 29,7 %. Este resultado confirma, con datos y no solo con diseño, la idea central del trabajo: separar la clasificación, la decisión y la justificación en componentes distintos —una política determinista que decide, un perfil de cliente que filtra, y un modelo de lenguaje que solo explica— permite obtener las ventajas de un modelo de lenguaje sin heredar su principal debilidad, que es decidir de forma no auditable.

La conclusión metodológica más relevante del trabajo se refiere a la medida de la explicabilidad: **un anclaje verificable perfecto no implica una justificación correcta**, y ambas propiedades deben medirse por separado. La verificación de anclaje comprueba que el modelo no inventa campos, y eso es necesario, pero se puede satisfacer sin explicar nada. Se documentaron cuatro maneras distintas de superarla vacíamente: describir al revés el significado de los campos citados, repetir el enunciado recibido —que contiene por construcción todos los campos—, atribuir a la alerta una técnica que procede de un pasaje recuperado y, la más problemática, contradecir la clasificación que el propio sistema acaba de emitir. Las dos últimas se cerraron con la misma regla: la clase y las técnicas son datos estructurados de la alerta, y el texto no puede desmentirlos. Una métrica automática de explicabilidad debe

defenderse de las formas de cumplirla sin cumplir su propósito.

De ahí se sigue una lectura más matizada de la recuperación aumentada que la que el trabajo esperaba encontrar. Medida por clase, la recuperación mejoraba con claridad la justificación de las amenazas reales y empeoraba la de los falsos positivos, hasta el punto de contradecir la clasificación que el propio motor acababa de emitir. La causa no estaba en la recuperación sino en el enunciado, que preguntaba «por qué importa esta alerta» incluso cuando el sistema acababa de decidir que no importa; corregido eso, la contradicción desaparece.

La corrección dejó, además, la mejor ilustración de la tesis anterior, y la dejó en tres tiempos. Al eliminar las justificaciones equivocadas, **la tasa de anclaje bajó de 1,00 a 0,61 mientras la calidad subía**, porque los falsos positivos pasaron de un texto fluido, anclado y falso a una descripción factual generada por plantilla. Y al dar a la recuperación el mismo tratamiento que al enunciado —fichas que explican por qué algo no es una amenaza, una consulta que depende de la clase decidida y un corpus partido en dos familias que no se mezclan— la tasa volvió a 1,00, esta vez con dieciocho textos leídos uno a uno y ninguno contradiciendo al motor. El mismo número, al principio y al final, midió dos situaciones opuestas. Una métrica que no distingue entre ellas no puede reportarse sola.

Dos conclusiones más se derivan de la evaluación de la recuperación. La primera es que el modelo de *embeddings* es el componente que fija el techo del conjunto: sustituir el modelo generativo por uno dedicado llevó la ficha correcta a estar siempre entre las cinco primeras, corrigió un hallazgo anterior —la consulta ampliada por el modelo pasó de recuperar peor a recuperar mejor que la fija— e hizo viable crecer el corpus, que con el modelo pequeño lo degradaba a la mitad. La segunda es que **la reproducibilidad de un modelo de lenguaje no la garantiza el muestreo determinista sino la infraestructura que lo sirve**: se midió que la caché de prefijos del servidor hacía que el mismo enunciado, a temperatura cero, produjera texto distinto según lo que el servidor hubiera procesado antes. Un requisito no funcional de reproducibilidad tiene que verificarse contra esa infraestructura, y así se hizo.

El trabajo deja, con la misma honestidad con que reporta sus logros, tres límites identificados y no resueltos. El primero afecta a la propia señal que hizo posible el resultado principal: la distinción entre un administrador legítimo y un atacante sobre el mismo servicio se apoya hoy en la dirección de origen declarada en la configuración del cliente, que es suplantable, y cubre cuatro de las seis categorías del caso de uso; las dos restantes exigen señales que el conjunto de datos no contiene. El segundo es que el clasificador con ajuste fino, que resolvería ese límite de raíz, permanece bloqueado porque el conjunto de datos disponible no ofrece variedad suficiente para entrenarlo sin invalidar la propia evaluación; pero el límite se acotó mejor de lo que se esperaba: cuando se generaron datos cuya verdad no venía de la heurística,

un árbol de decisión mostró que el aprendizaje supervisado aporta a esta escala exactamente una regla —la ráfaga pesa más que el origen declarado—, y esa regla se adoptó como código auditable, con lo que la señal de origen dejó de ser el único criterio y su suplantación dejó de ser invisible. El tercero es el alcance estadístico de la evaluación misma: cuarenta y dos alertas soportadas de tres familias, todas del mismo activo, sostienen un resultado direccionalmente claro pero no permiten generalizar sus cifras exactas a un cliente con un perfil de amenazas distinto. Las dos familias generadas al final del trabajo mostraron además que ampliar el conjunto de datos no es solo una cuestión de tamaño: la segunda hizo aflorar un falso positivo que la primera no podía revelar —el prototipo habría bloqueado a su propio auditor— y un límite de la explicación que ninguna instrucción al modelo corrige, porque hereda las etiquetas de la fuente; y la tercera mostró al método de referencia perdiendo amenazas, porque un único nivel numérico no puede separar a la vez el ruido de plataforma de una conexión en claro, y con ello la razón última por la que el prototipo no usa el nivel: decide con la familia, la postura del activo y el origen.

Recomendaciones

- **Ampliar el conjunto de datos con casos cuya verdad no venga de la heurística**, antes de intentar de nuevo el ajuste fino de un clasificador: más activos, más servicios, ataques lentos que no formen ráfaga, y usuarios legítimos desde orígenes no declarados. La campaña de casos difíciles y el árbol de decisión son el método: cada regla que el árbol descubra con evidencia estable se adopta como código; el codificador entrenado competirá contra ese árbol, no contra las reglas a secas.

- **Dar al reconocimiento una clase propia.** La familia no cambia la acción, y el diseño lo previó así; pero se midió que sí cambia la explicación: con la clase común y las etiquetas MITRE que Wazuh asigna a sus reglas de reconocimiento, un sondeo de puerto se narra como explotación aunque la familia y el conocimiento recuperado digan lo contrario. Una clase con su propio nombre, su propia prioridad —un sondeo aislado no merece la de una fuerza bruta— y su propia pregunta al justificador resuelve el problema en el único sitio donde puede resolverse. Se implementó tras cerrar la evaluación, en la variante que no mueve las cifras: el reconocimiento tiene ya clase y pregunta propias —la explicación deja de narrarlo como explotación— conservando la acción y la prioridad de un intento de acceso, de modo que la matriz y el escalado medidos no varían; rebajar además su prioridad, como aquí se sugiere, queda para cuando se acepte repetir la evaluación.

- **Crecer el corpus de descarte al ritmo de las clases, no antes.** Hoy contiene una ficha por cada clase sin amenaza, y basta porque las clases son tres. Cada motivo nuevo

de descarte que el perfil de cliente incorpore debe llegar acompañado de su ficha, porque se midió que sin ella la justificación del descarte cae a la plantilla. Con el *embedder* dedicado, crecer el corpus ya no degrada la recuperación de forma apreciable. Tras la evaluación se fijó ese acoplamiento como invariante verificado: cada clase sin amenaza declara la ficha que la fundamenta y una comprobación automática lo exige, de modo que el corpus solo puede crecer al ritmo de las clases y nunca antes.

- **Resolver, junto con Domotai C.A., si el sistema de logs propietario que originalmente motivó el proyecto llegará a estar disponible**, y en qué formato entrega sus eventos. De confirmarse, se integraría como una segunda fuente a través del mismo módulo de ingesta que hoy consume las alertas de Wazuh, sin desplazarlo.
- **Completar la validación sobre un equipo de borde con firmware real**, hoy sustituido en el laboratorio por un contenedor Linux provisional: las acciones del catálogo están diseñadas para ese firmware, pero solo se han verificado sobre su sustituto.
- **Someter la calidad de la justificación a una segunda revisión manual independiente** antes de dar por definitiva la lectura de utilidad reportada en este informe, dado que el método de evaluación de esa dimensión no es completamente automatizable.

Sobre el endurecimiento

Dos piezas de endurecimiento que el diseño dejaba para el despliegue se implementaron y verificaron al cierre: la traza encadenada por resumen criptográfico y el conector con usuario dedicado, clave y privilegios restringidos al catálogo. Ninguna cambia una cifra de la evaluación, y por eso se describen en la metodología y no en los resultados; pero las dos responden a la misma pregunta que recorre este trabajo, la de qué hace falta para que un analista pueda creer al sistema: que nadie pueda reescribir una decisión ya tomada, y que el sistema no pueda ejecutar en un equipo del cliente nada que no haya declarado poder ejecutar; y, con el anclaje de la traza en el propio Wazuh, que tampoco pueda hacer desaparecer una decisión borrando el final del registro. La rotación de la credencial del conector cierra el conjunto: todo o nada sobre todos los nodos, verificada antes de retirar la anterior, y anotada en el mismo sistema que las decisiones.

Sobre esa base, y ya fuera del prototipo evaluado, se ejecutó una hoja de ruta de endurecimiento para acercarlo a un despliegue real, recogida en una guía de integración y una hoja de ruta de cierre. En el propio motor en vivo: la reconstrucción del estado al reiniciar —la memoria de supresión y las contenciones vigentes se rederivan de la traza, de modo que un reinicio no olvida qué está conteniendo—; una señal de salud del daemon expuesta para su supervisión, con su unidad de servicio y reinicio automático ante caída; y una política

de retención de la traza que acota su crecimiento sin romper la cadena de verificación. En el control del operador: un interruptor que detiene toda la automatización al instante y un des-bloqueo en masa, para frenar y revertir si el sistema se equivoca. Y en la integración: la recepción de alertas por red con lista blanca de orígenes —en lugar de leer solo un fichero local—, la normalización de la zona horaria de los sucesos para que la correlación de ráfaga no dependa del huso del emisor, y una medida de la capacidad de un proceso que confirma que el cuello de botella no es el motor sino el ritmo del sistema de logs y el canal de respuesta. Como las anteriores, ninguna de estas piezas altera una cifra de la evaluación; y lo que queda para un cliente concreto —validar su conjunto de reglas, desplegar el auditor sobre su red y dimensionar el *hardware* del modelo— junto con las decisiones que corresponden a la organización —la autorización por escrito del alcance de actuación y la custodia de los secretos— se dejan identificadas en esa misma hoja de ruta.

Dos extensiones más, también posteriores a la evaluación, ensanchan el alcance sin tocar sus cifras. El registro de familias resultó extensible en la práctica: partiendo de las cuatro familias del perímetro evaluado, se incorporaron seis más —hasta diez— que el motor atiende con el mismo catálogo cerrado de acciones. La quinta —escalada y manipulación de cuentas (uso de `sudo`, alta y cambio de cuentas)— responde bloqueando la propia cuenta (forzar el cambio de contraseña), de validación humana incondicional; las demás estrenan respuestas dirigidas a la forma del ataque: aislar en cuarentena el host comprometido sobre el cortafuegos de borde (cambios de integridad de ficheros y detección de malware), acotar la banda del enlace ante una inundación (denegación de servicio), cortar la sesión viva de un acceso ya consumado, y bloquear en el borde el egreso hacia un destino malicioso conocido (exfiltración y mando y control). Todas se ejercitaron en vivo sobre el laboratorio y ninguna pertenece al conjunto de datos medido, de modo que ensanchan la cobertura sin alterar las cifras de la evaluación. Y la validación humana, que el prototipo evaluado hacía por terminal, se llevó además a un **visor web**: una consola de operaciones con el flujo de decisiones, la cola de aprobaciones —con aviso al analista (notificación y señal sonora al entrar una decisión) y el tiempo de espera de cada una a la vista, como un acuerdo de nivel de servicio—, el **ciclo de vida del caso** (estado, responsable y notas del analista, anotados como registros encadenados en la misma traza auditable), una matriz de cobertura MITRE ATT&CK, el grafo del incidente sobre el mapa de la red y un informe exportable, desde la que el analista aprueba, pausa la automatización o revierte una contención sin salir del navegador.

El propio sistema se sometió además a una auditoría de seguridad —un *pentest* del MDR tratándolo como si estuviera en producción— que enumeró diecinueve puntos y motivó una tanda de endurecimiento posterior, también sin alterar las cifras de la evaluación. De ella salieron la firma **HMAC** opcional de la cadena de la traza (para que quien pueda reescribir el fichero no pueda recalcular una cadena válida sin la clave), el conector *fail-closed* (no

ejecuta nada sin credencial), el blindaje de la consola contra peticiones forzadas entre sitios (validación de `Host` y `Origin`, CORS restringido) y, de forma destacada, una **autenticación opcional de la consola** por *token* con no repudio del operador: la consola, ligada únicamente a `127.0.0.1`, no exigía credencial, y ahora puede hacerlo cuando el cliente lo active. Ninguna de estas piezas cambia una decisión del motor; todas responden a la misma pregunta que recorre el trabajo —qué hace falta para que un analista pueda creer al sistema— y, con la misma honestidad, nombran lo que todavía falta: el inicio de sesión corporativo y el cifrado del canal son parte del despliegue del cliente.

Trabajo futuro

Quedan fuera del alcance de esta pasantía, y se documentan explícitamente para que no se interpreten como un olvido: la respuesta automatizada sin validación humana previa, que el diseño actual descarta de forma deliberada; la integración multicapa con fuentes de identidad, correo y nube, que ampliaría la correlación más allá de la red del cliente; el aprendizaje continuo a partir de la retroalimentación que el analista ya registra en cada validación, hoy almacenada pero no utilizada para ajustar ningún modelo; y el servicio simultáneo de varios clientes desde una sola instancia —multi-inquilino—, del que el empaquetado, la supervisión y la medida de capacidad de un proceso ya quedaron resueltos, pero no el reparto de varios perfiles dentro del mismo. Cada una de estas líneas es una extensión natural sobre la arquitectura ya construida, y ninguna exige rediseñarla desde cero.

Referencias

[1] Ehsan Aghaei, Xi Niu, Waseem Shadid, and Ehab Al-Shaer. SecureBERT: A domain-specific language model for cybersecurity. *arXiv preprint*, 2022. arXiv:2204.02685.

[2] Cisco Foundation AI. Foundation-Sec-8B. https://huggingface.co/fdtn-ai/Foundation-Sec-8B, 2025. Ficha del modelo; consultada el 2 de septiembre de 2026.

[3] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020. arXiv:2005.11401.

[4] OWASP Foundation. OWASP top 10 for LLM applications. https://genai.owasp.org/llm-top-10/, 2025. Consultado el 25 de agosto de 2026.

[5] Containerlab. Containerlab — documentación oficial. https://containerlab.dev/, 2026. Consultado el 23 de agosto de 2026.

[6] MITRE Engenuity. MITRE ATT&CK Evaluations — enterprise. https://attackevals.mitre-engenuity.org/, 2025. Resultados públicos de detección por plataforma; consultado el 25 de agosto de 2026.

[7] MITRE Corporation. MITRE ATT&CK. https://attack.mitre.org/, 2026. Consultado el 25 de agosto de 2026.

[8] MITRE Corporation. MITRE ATLAS. https://atlas.mitre.org/, 2026. Panorama de amenazas contra sistemas de inteligencia artificial; consultado el 25 de agosto de 2026.

[9] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *arXiv preprint*, 2023. arXiv:2307.03172.

[10] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR 2023)*, 2023. arXiv:2210.03629.

[11] Meta AI. CyberSecEval: A secure coding benchmark for language models. *arXiv preprint*, 2023. arXiv:2312.04724 — publicado como parte de la iniciativa Purple Llama.